

Introduction to Data Science and Engineering

- R programming



Some materials courtesy of
Rafael A. Irizarry, and are modified
from the original version.

Zhenqin (Michael) Wu / 吳楨欽

School of Computing and Data Science
University of Hong Kong

Slide deck originally created by RB Luo



In this lecture

- Introduction to R and RStudio
- Basics of R programming
- Data types & data structures
- Basic operations



In this lecture

- **Introduction to R and RStudio**
- Basics of R programming
- Data types & data structures
- Basic operations

The two sides of R programming

It's easy to learn



It's difficult to master





Why R?

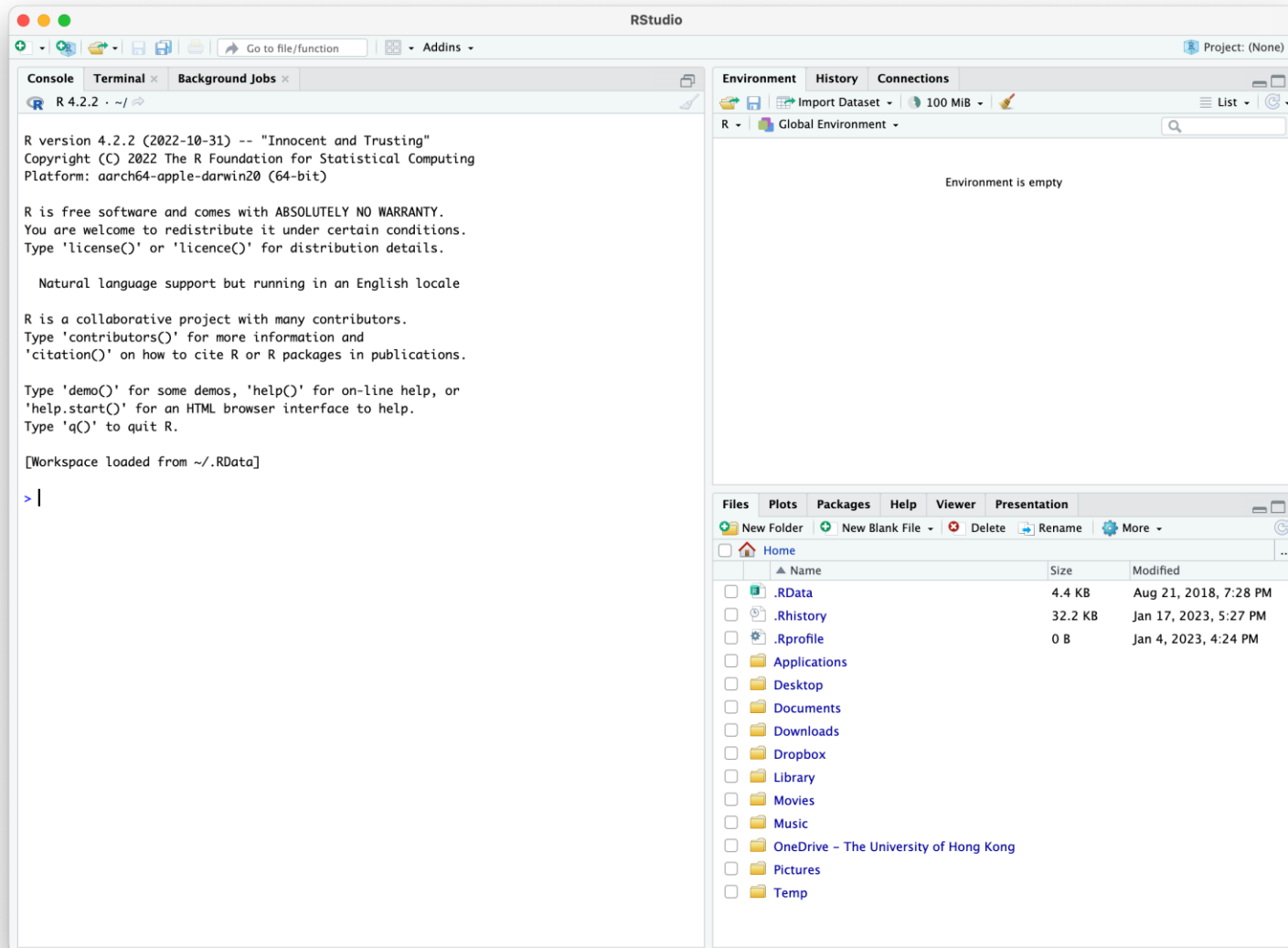
- **R: specialized language for interactive data analysis**
- **Interactivity:**
 - Read-Evaluate-Print Loop (REPL), line-by-line execution
 - Data inspection, visualization, plotting on-the-fly
- **Reproducibility:**
 - Scripts for data analysis routines
- **Statistical strengths:**
 - Built-in functions and diverse packages for statistical modeling, hypothesis testing, etc.
- **Data visualization:**
 - Best-in-class visualization tool (e.g., `ggplot2`), intuitive graph design



Other attractive features of R

- Free and open source
- Run on all major platforms: Windows, MacOS, UNIX/Linux
- Easy-to-follow tutorials via R vignettes
- Large, growing, and active community of R users, numerous resources for learning and asking questions
- Widespread usage in statistical research and many new data science domains: finance, ecology, molecular biology, social sciences, geography ...

Introduction to Studio[®]



Install R and RStudio

- Install R (the programming language) at: <https://cran.r-project.org/>



CRAN
[Mirrors](#)
[What's new?](#)
[Search](#)
[CRAN Team](#)

About R
[R Homepage](#)

The Comprehensive R Archive Network

Download and Install R

Precompiled binary distributions of the base system and contributed packages,
Windows and Mac users most likely want one of these versions of R:

- [Download R for Linux](#) ([Debian](#), [Fedora/Redhat](#), [Ubuntu](#))
- [Download R for macOS](#)
- [Download R for Windows](#)

R is part of many Linux distributions, you should check with your Linux package management system in addition to the link above.

- Special notes for Mac users

For Apple silicon (M1,2,..) Macs:

[R-4.5.1-arm64.pkg](#)

SHA1-hash: 0db802faf0e544168794a6d648c73a48c2b51a5d
(ca. 97MB, notarized and signed)

For older Intel Macs:

[R-4.5.1-x86_64.pkg](#)

SHA1-hash: 5384a1b3458a28030fc043e64c113e3af40f4c58
(ca. 100MB, notarized and signed)



Install R and RStudio

- Install R Studio (Integrated Development Environment, IDE) at: <https://posit.co/downloads/>

DOWNLOAD

RStudio IDE

The most popular coding environment for R, built with love by Posit.

Used by millions of people weekly, the RStudio integrated development environment (IDE) is a set of tools built to help you be more productive with R and Python. It includes a console, syntax-highlighting editor that supports direct code execution. It also features tools for plotting, viewing history, debugging and managing your workspace.

If you're a professional data scientist and want guidance on adopting open source in your organization, don't hesitate to [book a call with us.](#)

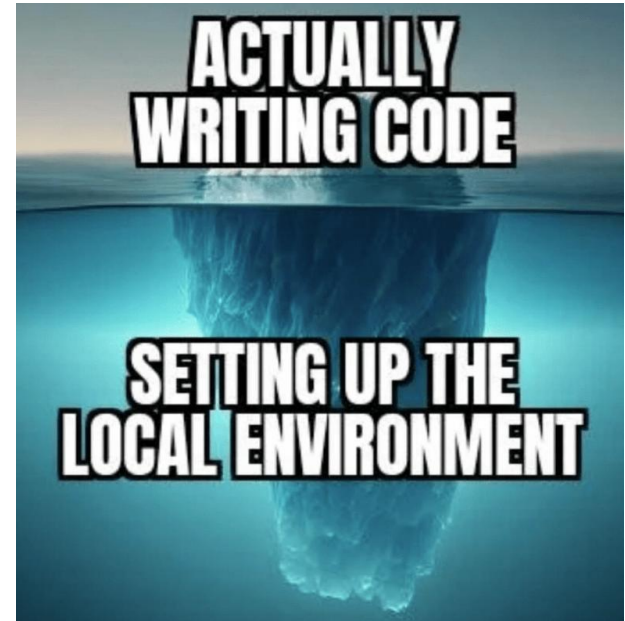
DOWNLOAD RSTUDIO

DOWNLOAD RSTUDIO SERVER

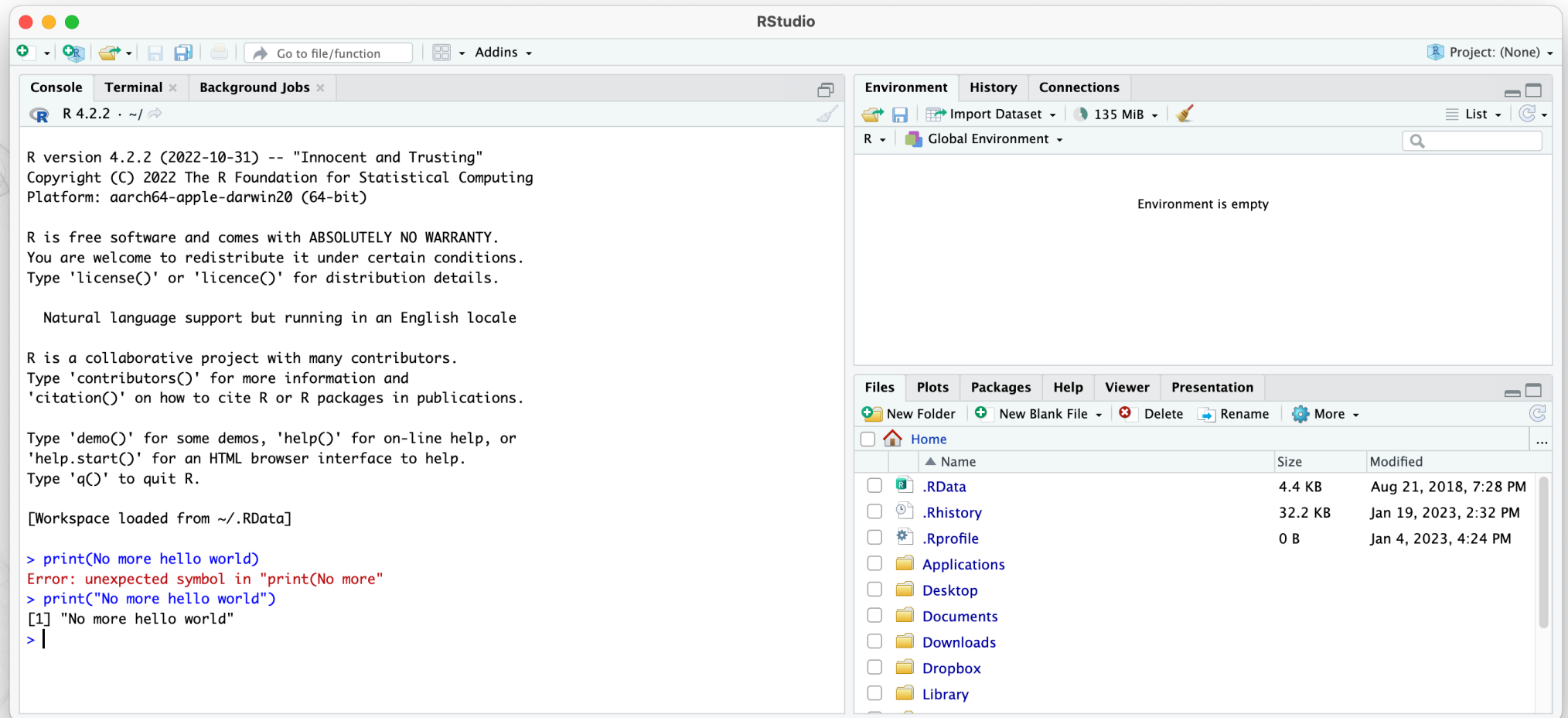


Installation and environment setup

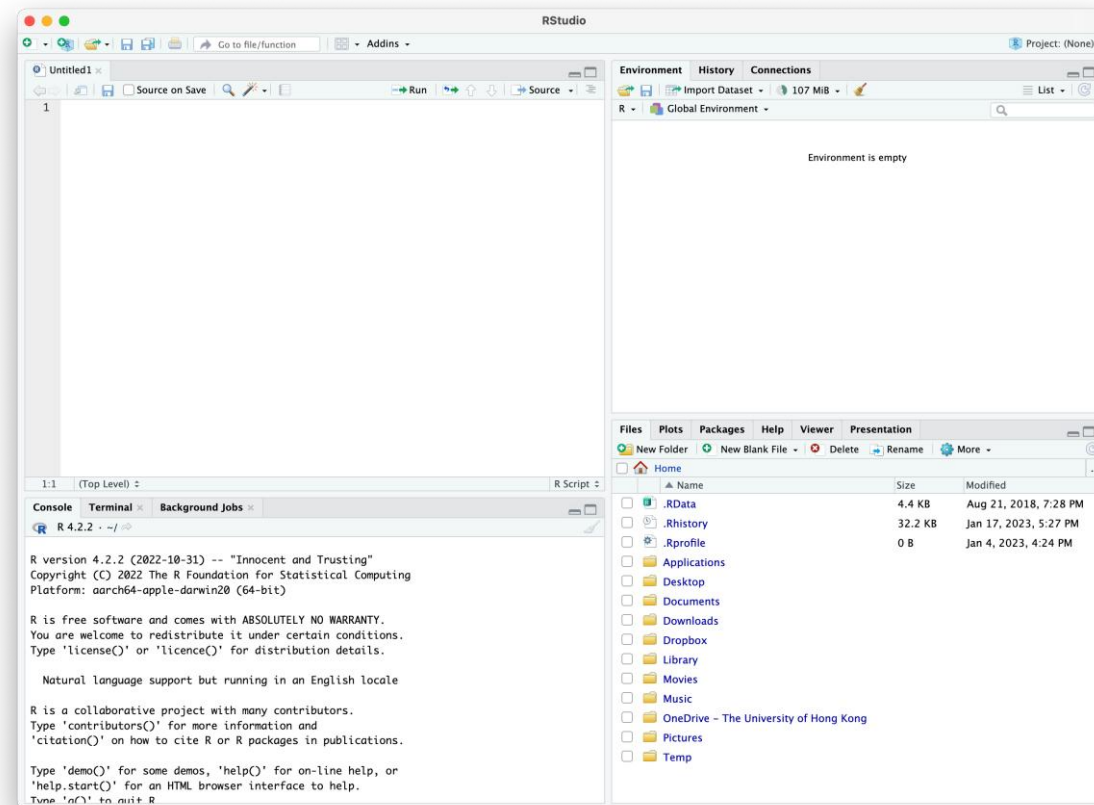
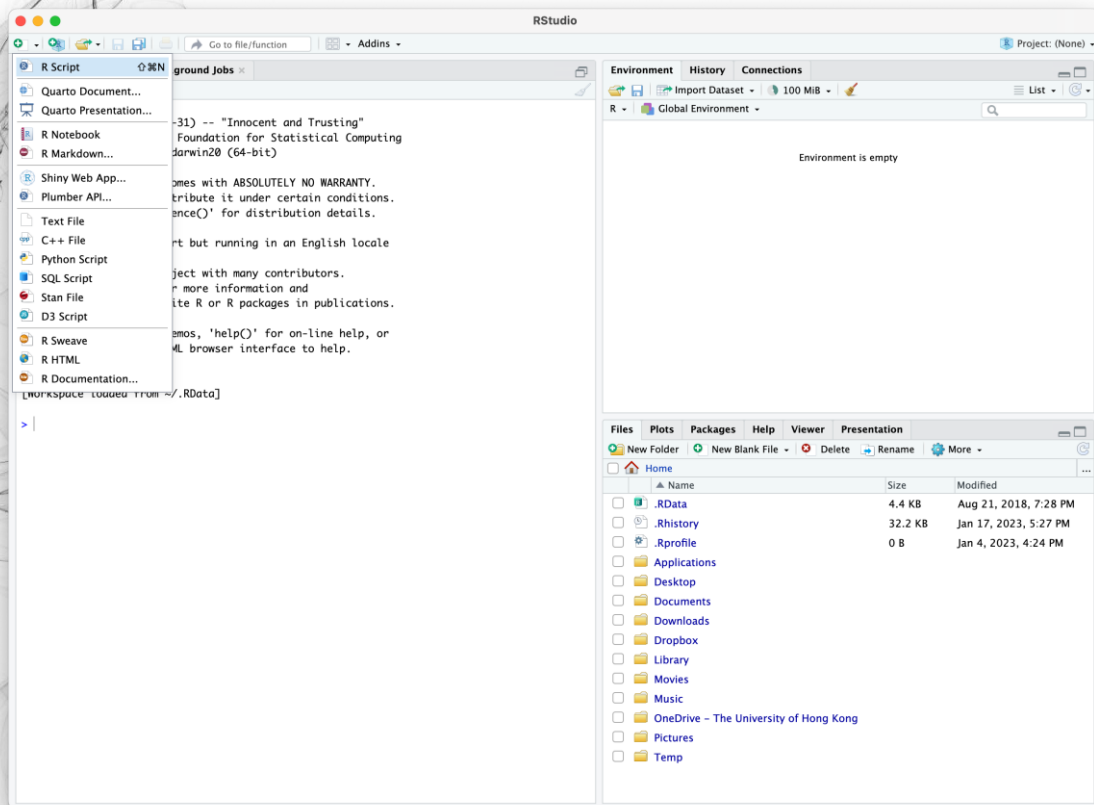
- Any problems? (priority list for seeking help)
 - 1st: Google
 - 2nd: GPT, DeepSeek, LLMs, etc.
 - 3rd: Your peers and friends
 - 4th: TAs and instructor, through email, f2f, zoom, etc.
- Environment setup and management
 - Know your context: OS, hardwares
 - Trial and error
 - R is actually simpler



Ritual



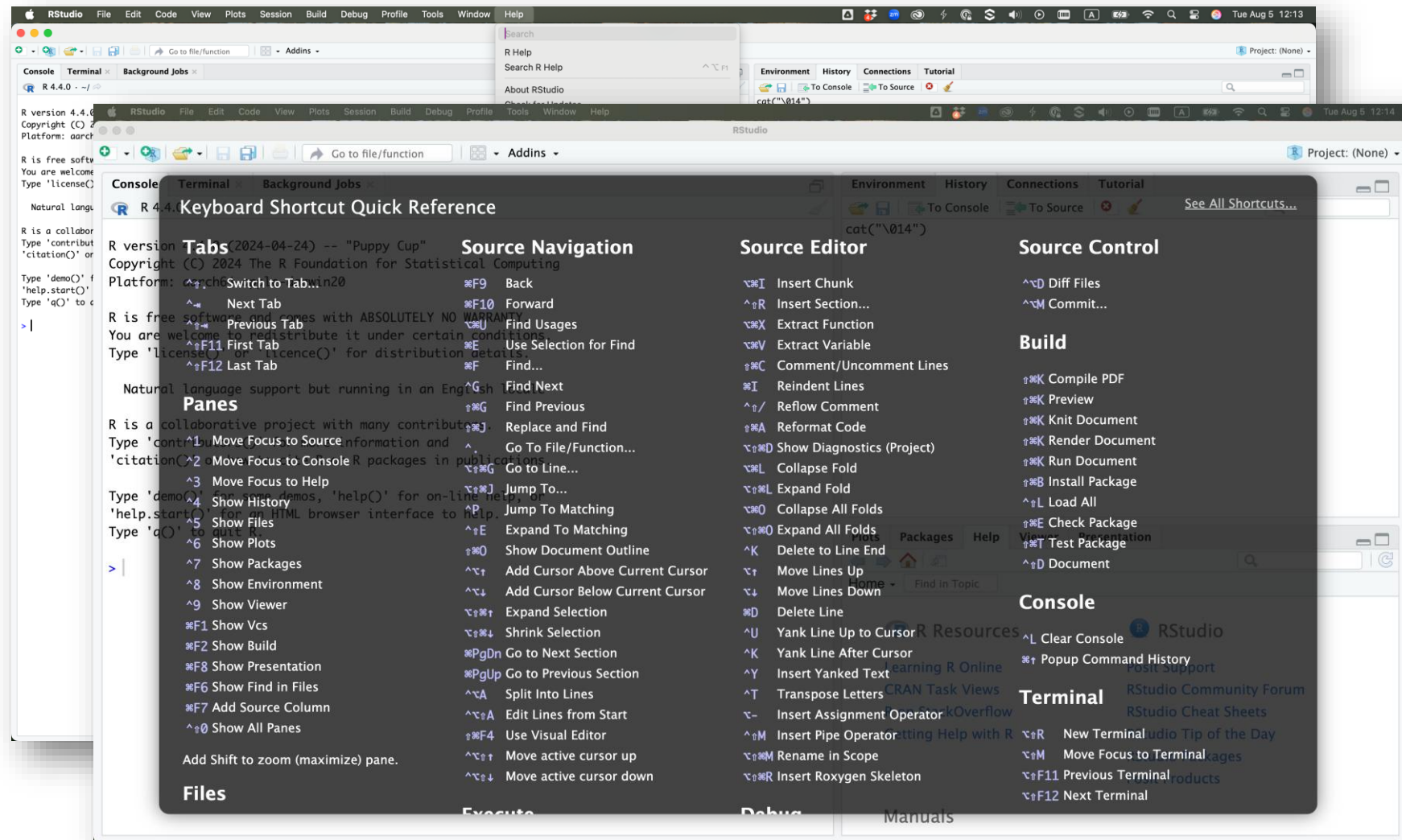
Create a script



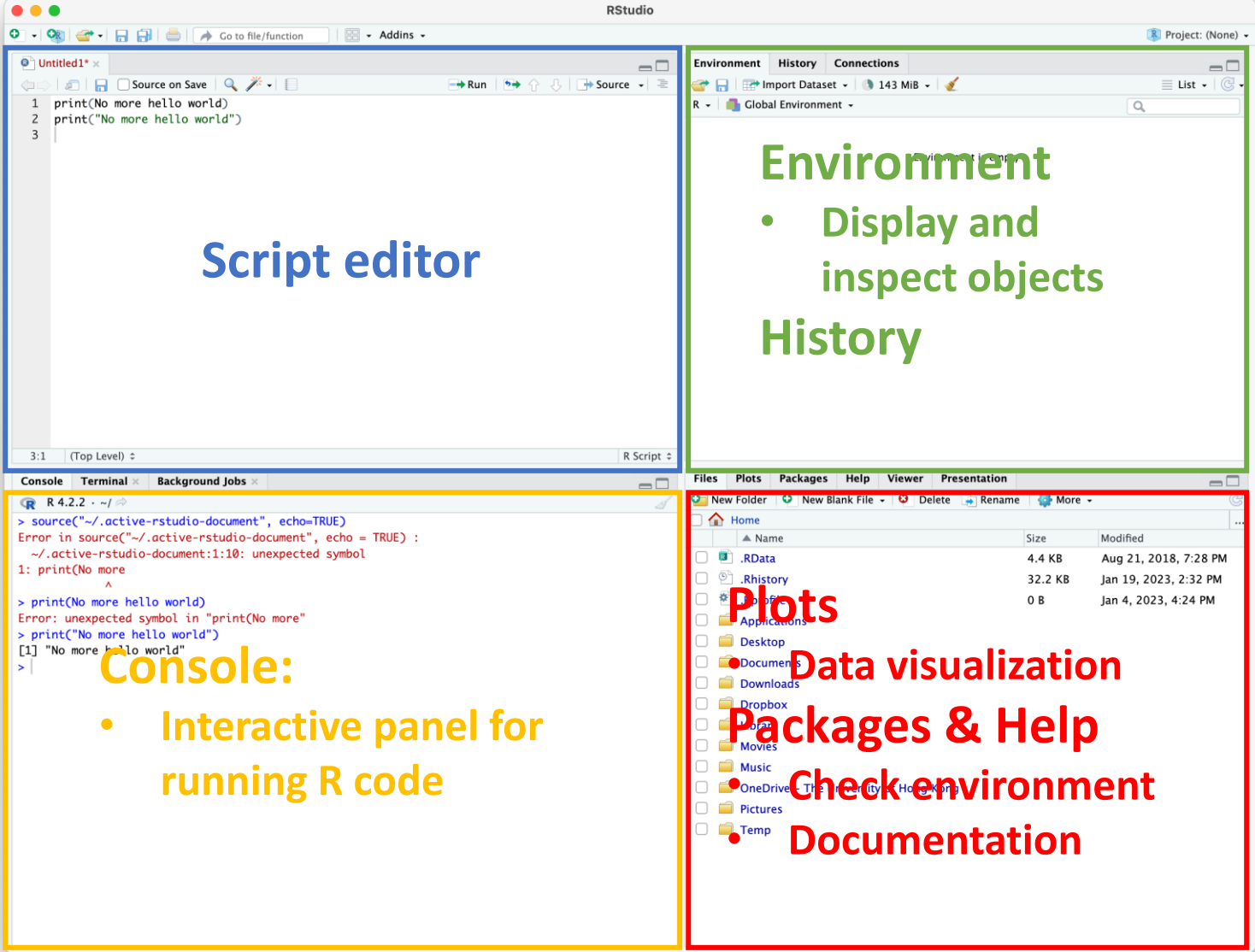
Windows: **Ctrl + Shift + N**

Mac: **Cmd + Shift + N**

Use shortcuts



Your workspace



The screenshot shows the RStudio interface with four main panels highlighted by colored boxes and labels:

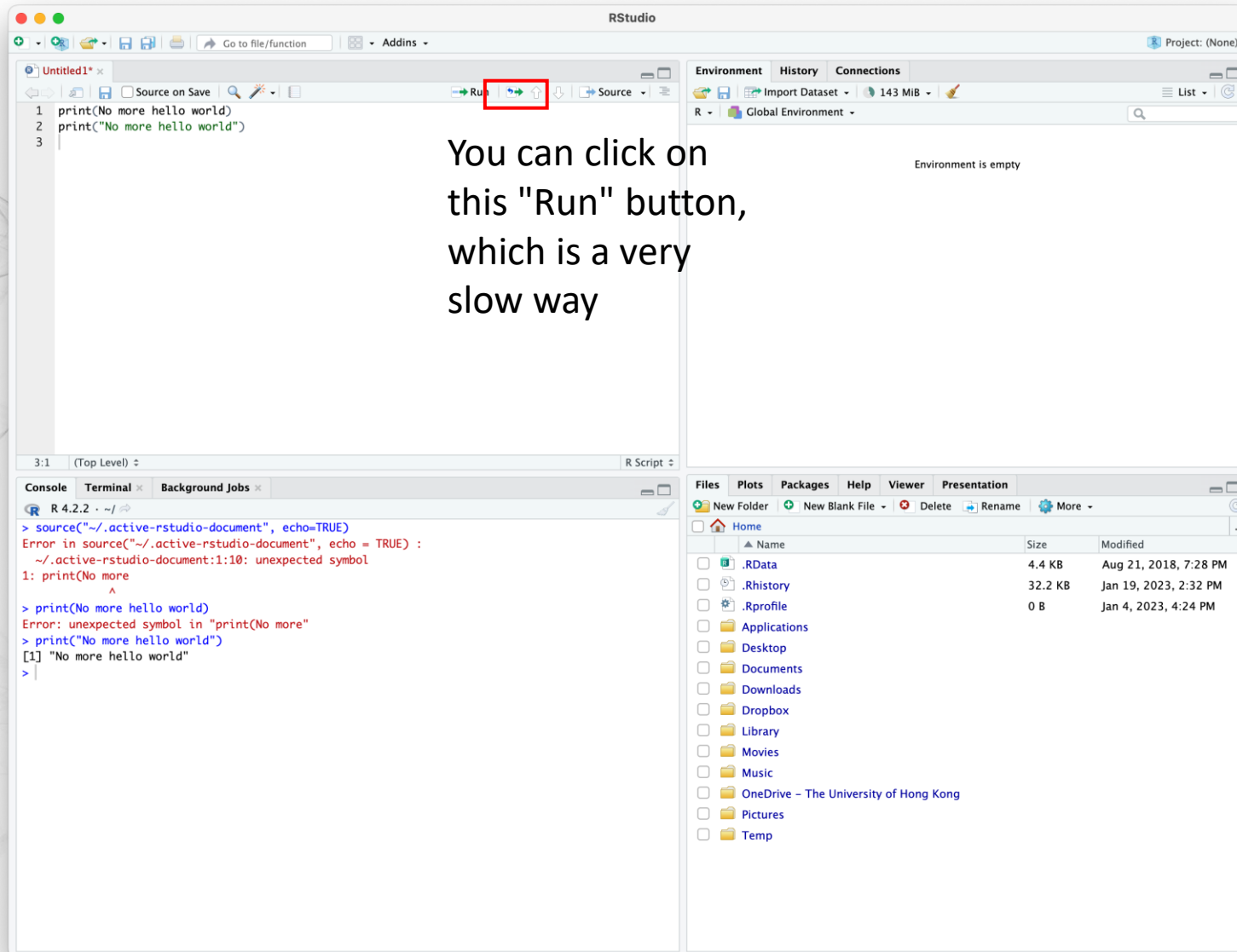
- Script editor** (blue box): The top-left panel containing an R script file named "Untitled1.R" with the following code:

```
1 print("No more hello world")
2 print("No more hello world")
3
```
- Environment** (green box): The top-right panel, labeled "Environment", "History", and "Connections". It displays the "Global Environment" with a memory usage of 143 MiB. The label "Display and inspect objects" is also present.
- Console** (yellow box): The bottom-left panel showing the R console output and error messages. The output includes:

```
> source("~/active-rstudio-document", echo=TRUE)
Error in source("~/active-rstudio-document", echo = TRUE) :
  ~/active-rstudio-document:1:10: unexpected symbol
1: print("No more
  ^
> print("No more hello world")
Error: unexpected symbol in "print("No more"
> print("No more hello world")
[1] "No more hello world"
>
```
- Plots** (red box): The bottom-right panel, labeled "Plots", "Packages", "Help", "Viewer", and "Presentation". It shows a file explorer view of the home directory with a table of files and folders. The label "Data visualization" is also present.

Name	Size	Modified
.RData	4.4 KB	Aug 21, 2018, 7:28 PM
.Rhistory	32.2 KB	Jan 19, 2023, 2:32 PM
.Rproj	0 B	Jan 4, 2023, 4:24 PM
Applications		
Desktop		
Documents		
Downloads		
Dropbox		
Music		
OneDrive - The University of Hong Kong		
Pictures		
Temp		

Ritual - Continued



Run the current line:

Mac: **Cmd + Enter**

Win: **Ctrl + Enter**

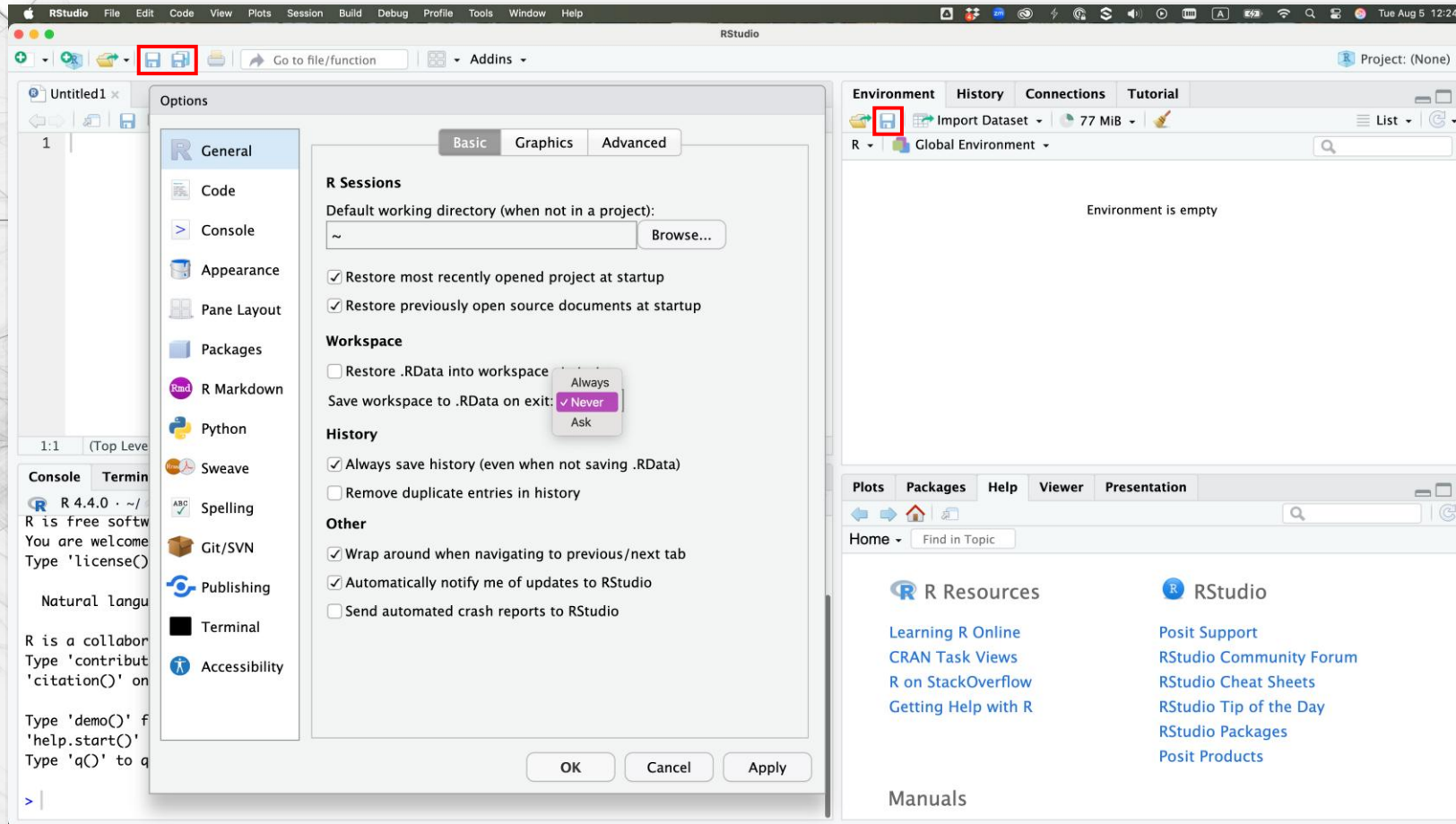
Run the whole script:

Mac: **Cmd + Shift + Enter**

Win: **Ctrl + Shift + Enter**

Change global options

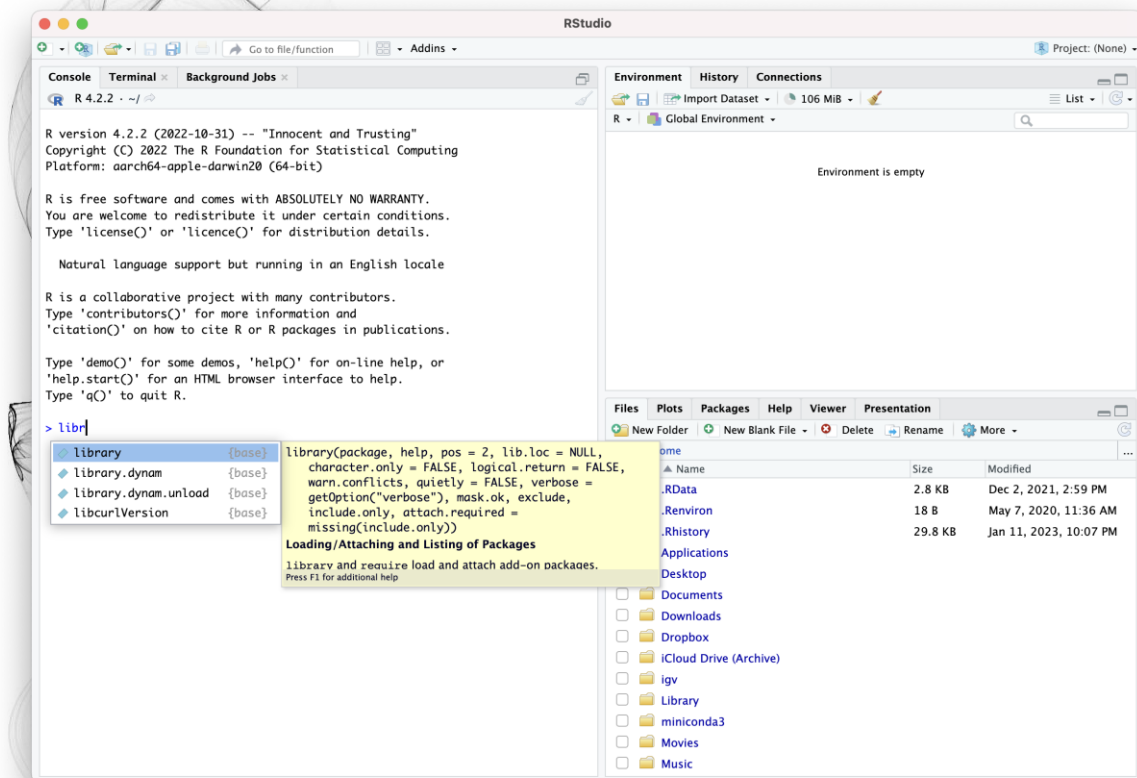
In tools > global options > general



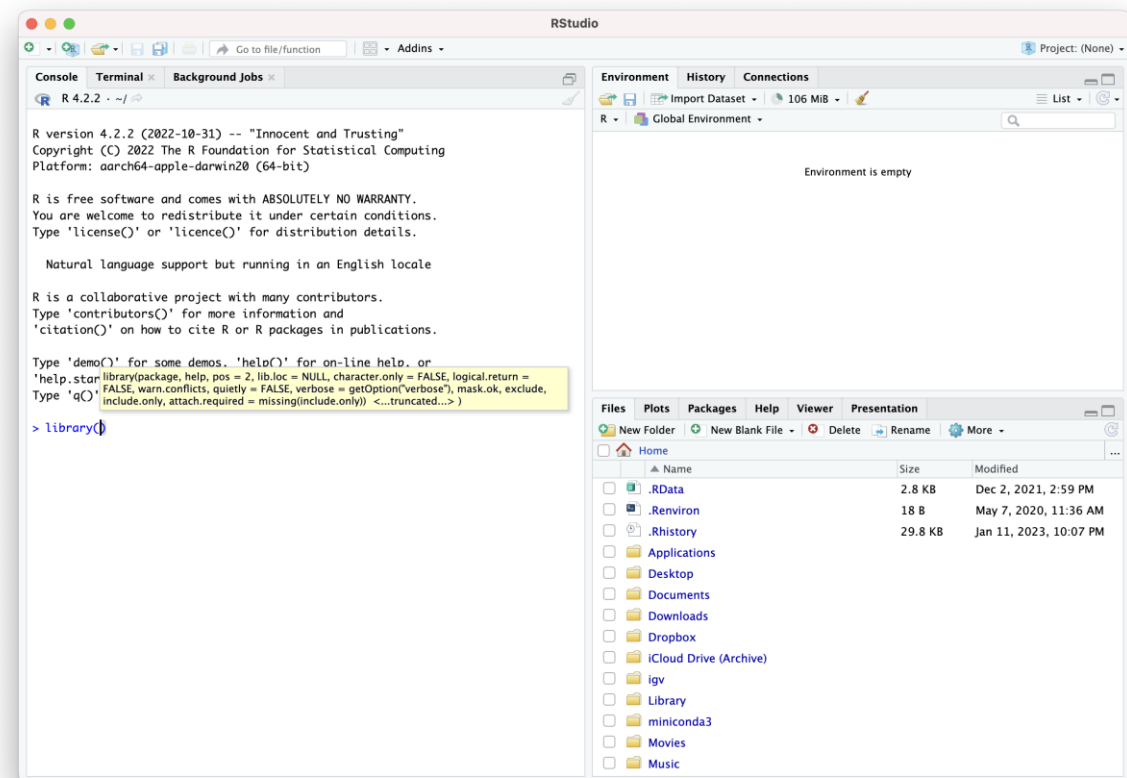
Loading previous session's data is prone to unexpected problems.

- Save your scripts frequently
- Save your workspace when necessary

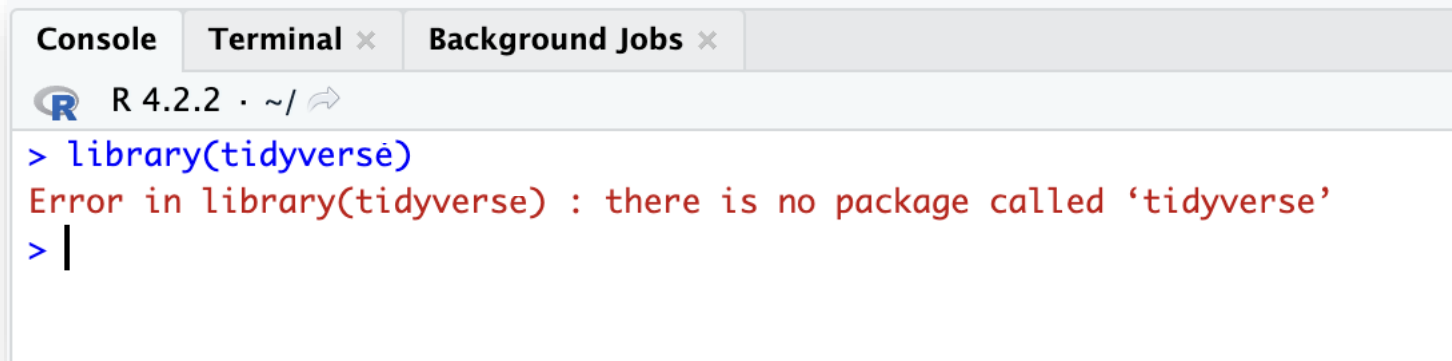
Load a package



tab



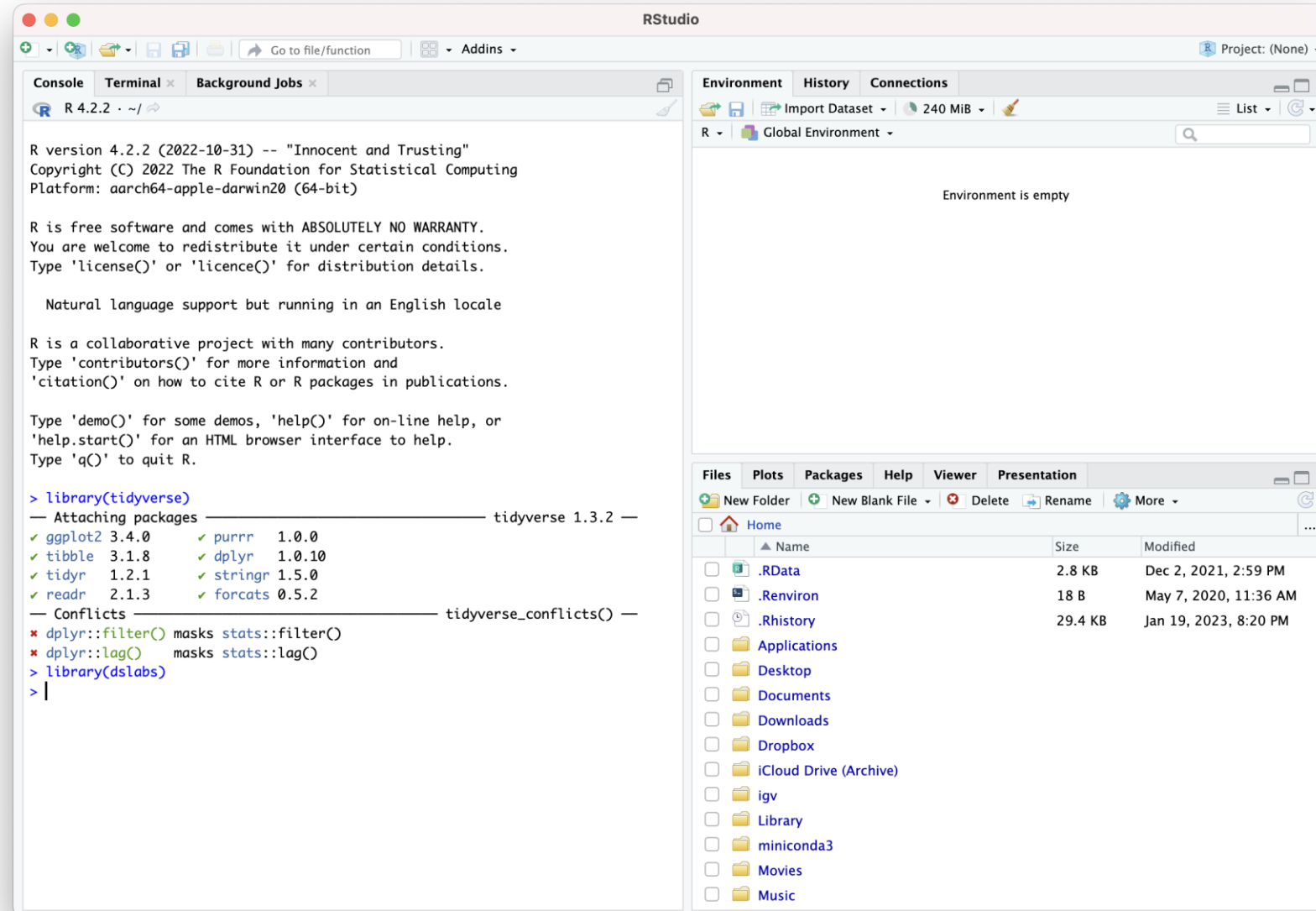
Load a package

A screenshot of an R console window. The window has three tabs: 'Console', 'Terminal', and 'Background Jobs'. The 'Console' tab is active. The prompt is 'R 4.2.2 · ~/'. The user has entered the command '> library(tidyverse)'. The console displays an error message in red text: 'Error in library(tidyverse) : there is no package called \'tidyverse\''. The prompt '> |' is shown on the next line.

```
Console Terminal × Background Jobs ×
R 4.2.2 · ~/ ↵
> library(tidyverse)
Error in library(tidyverse) : there is no package called 'tidyverse'
> |
```

- **Install packages:**
`install.packages("tidyverse")`
Also, please install "dslabs"
- **Better version control:**
`install.packages("remotes")`
`library(remotes)`
`install_version("tidyverse", "1.3.2")`

Load a package



Keep track of your environment

Files

Plots

Packages

Help

Viewer

Presentation

Install

Update

</



In this lecture

- Introduction to R and RStudio
- **Basics of R programming**
- Data types & data structures
- Basic operations

Objects

To solve a quadratic equation

$$x^2 + x - 1 = 0,$$

Analytic expression of the solutions

$$\frac{-b - \sqrt{b^2 - 4ac}}{2a} \quad \text{and} \quad \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

First, we define the three variables

```
Console Terminal x Background Jobs x
R 4.2.2 · ~/ ↻
> a <- 1
> b <- 1
> c <- -1
>
```

Using = instead of <- for assignment



We use <- to **assign** values to the variables.

Compare to python:

```
a = 1
b = 1
c = -1
```

= also works in R, but is less recommended.

And there is ->

NO, not recommended

Objects

After defining them, they should appear in **Environment** page



Environment		History	Connections
R		Global Environment	
Values			
a	1		
b	1		
c	-1		



Objects

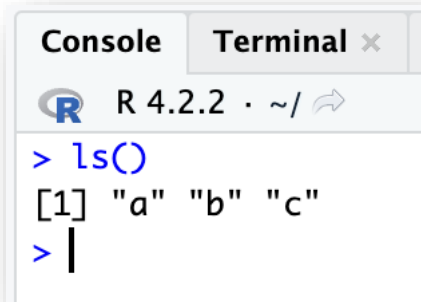
Print the variables

```
> a  
[1] 1  
> b  
[1] 1  
> c  
[1] -1  
> print(a)  
[1] 1  
> print(b)  
[1] 1  
> print(c)  
[1] -1  
>
```

We use the term *object* to describe stuff that is stored in R. Variables are examples, but objects can also be more complicated entities such as functions and data frames which are described later.

Work with objects

As we define objects in the console, we are actually changing the workspace. You can see all the variables saved in your workspace by typing:

A screenshot of the R console window. The title bar shows 'Console' and 'Terminal x'. The R logo and version 'R 4.2.2 · ~/ ' are visible. The prompt is '>'. The command 'ls()' has been entered, and the output is '[1] "a" "b" "c"'. The prompt is now '> |' with a cursor.

```
Console Terminal x
R 4.2.2 · ~/ 
> ls()
[1] "a" "b" "c"
> |
```

`ls` is a function, you need parentheses to “call” it.
What if we missed the `()`?

```
> x
Error: object 'x' not found
```

Visiting an undefined object causes error

```
> True
Error: object 'True' not found
```

Case-sensitive: `This` $\neq$ `THIS` $\neq$ `this`
`TRUE` is the logical value, `True` is not



Work with objects

```
> (-b + sqrt(b^2 - 4*a*c) ) / ( 2*a )  
[1] 0.618034  
> (-b - sqrt(b^2 - 4*a*c) ) / ( 2*a )  
[1] -1.618034
```

In R, square root function `sqrt` is built in.

A list of R built-in functions: <https://www.datacamp.com/doc/r/functions>

Variable naming

Shave Hours Off Any Project



Variable Naming

The hardest part of coding

O RLY?

Creative Var. Name

R programming style guide
<http://adv-r.had.co.nz/Style.html>

- Bad examples

- `a <- (-b + sqrt(b^2 - 4*a*c)) / (2*a)`
- `b <- (-b - sqrt(b^2 - 4*a*c)) / (2*a)`

- Good examples

- `solution_1 <- (-b + sqrt(b^2 - 4*a*c)) / (2*a)`
- `solution_2 <- (-b - sqrt(b^2 - 4*a*c)) / (2*a)`

- VariableNameAsAComment style

- `OnJan15thIn2024WeGatheredAtRHTAndWitnessedSomeRidiculousPie`

- General recommendation: `snake_case`

- Be descriptive
- all lower case
- `use_underscore_not_space`

Functions

Another built-in function

```
> log(10)
[1] 2.302585
> log(a)
[1] 0
>
```

How to do log2 or log10?

```
log(x, base=2)
log(x, base=10)
```

To find out more about a function, use:

```
> help("log")
> ?log
> |
```



The screenshot shows the R Documentation window for the topic 'Logarithms and Exponentials'. The window has a menu bar with 'Files', 'Plots', 'Packages', 'Help', 'Viewer', and 'Presentation'. Below the menu bar is a search bar and a 'Find in Topic' button. The main content area is titled 'Logarithms and Exponentials' and includes a 'Description' section, a 'Usage' section, and an 'Arguments' section. The 'Description' section explains that `log` computes logarithms, with `log10` for common (base 10) and `log2` for binary (base 2) logarithms. The 'Usage' section lists the functions `log(x, base = exp(1))`, `logb(x, base = exp(1))`, `log10(x)`, `log2(x)`, `log1p(x)`, `exp(x)`, and `expm1(x)`. The 'Arguments' section defines `x` as a numeric or complex vector and `base` as a positive or complex number, defaulting to $e = \exp(1)$.

R Documentation

Logarithms and Exponentials

Description

`log` computes logarithms, by default natural logarithms, `log10` computes common (i.e., base 10) logarithms, and `log2` computes binary (i.e., base 2) logarithms. The general form `log(x, base)` computes logarithms with base `base`.

`log1p(x)` computes $\log(1 + x)$ accurately also for $|x| \ll 1$.

`exp` computes the exponential function.

`expm1(x)` computes $\exp(x) - 1$ accurately also for $|x| \ll 1$.

Usage

```
log(x, base = exp(1))
logb(x, base = exp(1))
log10(x)
log2(x)

log1p(x)

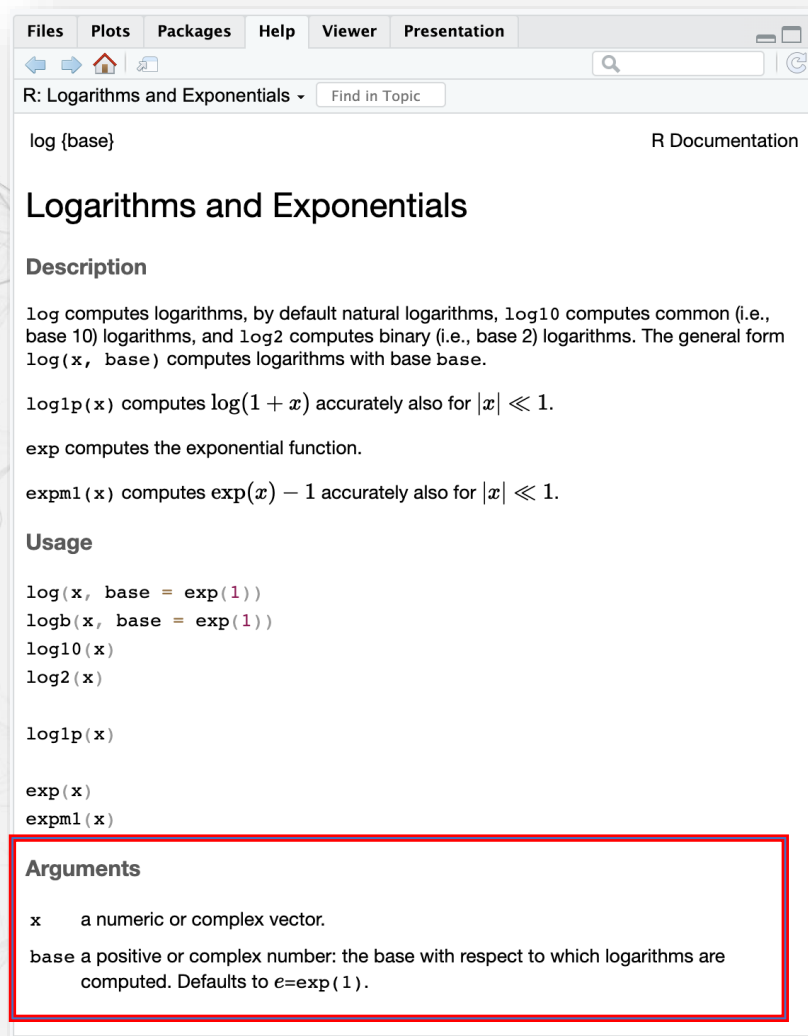
exp(x)
expm1(x)
```

Arguments

x a numeric or complex vector.

base a positive or complex number: the base with respect to which logarithms are computed. Defaults to $e = \exp(1)$.

Functions



The help manual of `log` shows that function `log` needs: `x`, `base`

However, some arguments are required, and others are optional

You can determine which arguments are optional by noting in the help document that a default value is assigned with `=`

Optional arguments have default values. For example, `base` of the function `log` defaults to `base = exp(1)`, which is natural log



Functions

```
> log(8,2)
[1] 3
> log(8,base=2)
[1] 3
> log(x=8,base=2)
[1] 3
> log(base=2,x=8)
[1] 3
> log(2,8)
[1] 0.3333333
> |
```

You either:

- Stick to the sequence
- Specify the argument

Proper styles in R:

- `<-` is only used for assigning values
- `=` used for providing values to arguments

```
> x=8
> log(x,base=2)
[1] 3
> log(x=16,base=2)
[1] 4
> x
[1] 8
> log(x<-16,base=2)
[1] 4
> x
[1] 16
```



Functions

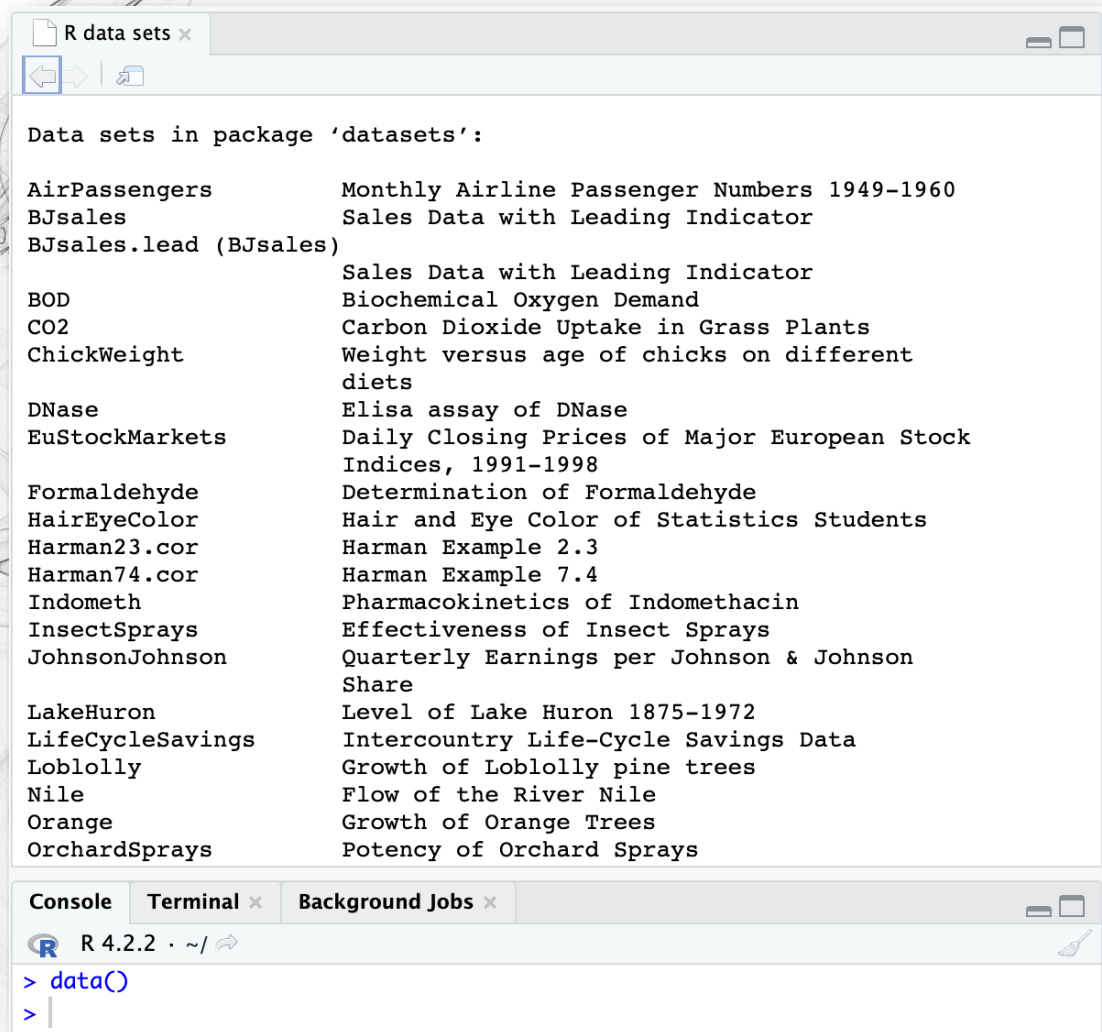
```
> ?"+"  
> ?">"  
> ?"^"
```

Find helps on arithmetic and relational operators. Remember to include the quotes

```
> a<-1  
> rm(a)  
> a  
Error: object 'a' not found  
>
```

`rm` to remove an object

Work with built-in datasets



The screenshot shows an R console window with the title 'R data sets'. The main pane displays the contents of the 'datasets' package as a list of data sets with their descriptions. At the bottom, the console shows the command `> data()` being entered.

```
R data sets x
Data sets in package 'datasets':

AirPassengers      Monthly Airline Passenger Numbers 1949-1960
BJsales            Sales Data with Leading Indicator
BJsales.lead (BJsales) Sales Data with Leading Indicator
BOD                Biochemical Oxygen Demand
CO2                Carbon Dioxide Uptake in Grass Plants
ChickWeight        Weight versus age of chicks on different diets
DNase              Elisa assay of DNase
EuStockMarkets     Daily Closing Prices of Major European Stock Indices, 1991-1998
Formaldehyde       Determination of Formaldehyde
HairEyeColor       Hair and Eye Color of Statistics Students
Harman23.cor       Harman Example 2.3
Harman74.cor       Harman Example 7.4
Indometh           Pharmacokinetics of Indomethacin
InsectSprays       Effectiveness of Insect Sprays
JohnsonJohnson    Quarterly Earnings per Johnson & Johnson Share
LakeHuron          Level of Lake Huron 1875-1972
LifeCycleSavings   Intercountry Life-Cycle Savings Data
Loblolly           Growth of Loblolly pine trees
Nile               Flow of the River Nile
Orange             Growth of Orange Trees
OrchardSprays      Potency of Orchard Sprays

Console  Terminal x  Background Jobs x
R 4.2.2 · ~/
> data()
>
```

PlantGrowth	Results from an Experiment on Plant Growth
Puromycin	Reaction Velocity of an Enzymatic Reaction
Seatbelts	Road Casualties in Great Britain 1969-84
Theoph	Pharmacokinetics of Theophylline
Titanic	Survival of passengers on the Titanic
ToothGrowth	The Effect of Vitamin C on Tooth Growth in Guinea Pigs
UCBAdmissions	Student Admissions at UC Berkeley
UKDriverDeaths	Road Casualties in Great Britain 1969-84
UKgas	UK Quarterly Gas Consumption
USAccDeaths	Accidental Deaths in the US 1973-1978
USArrests	Violent Crime Rates by US State
USJudgeRatings	Lawyers' Ratings of State Judges in the US Superior Court
USPersonalExpenditure	Personal Expenditure Data
UScitiesD	Distances Between European Cities and Between US Cities
VADeaths	Death Rates in Virginia (1940)
WWWusage	Internet Usage per Minute
WorldPhones	The World's Telephones
ability.cov	Ability and Intelligence Tests
airmiles	Passenger Miles on Commercial US Airlines, 1937-1960
airquality	New York Air Quality Measurements
anscombe	Anscombe's Quartet of 'Identical' Simple Linear Regressions
attenu	The Joyner-Boore Attenuation Data
attitude	The Chatterjee-Price Attitude Data
austres	Quarterly Time Series of the Number of Australian Residents
beaver1 (beavers)	Body Temperature Series of Two Beavers
beaver2 (beavers)	Body Temperature Series of Two Beavers
cars	Speed and Stopping Distances of Cars

Many more: `data(package = .packages(all.available = TRUE))` 33

Work with built-in datasets

```
> data("co2")
> co2
```

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
1959	315.42	316.31	316.50	317.56	318.13	318.00	316.39	314.65	313.68	313.18	314.66	315.43
1960	316.27	316.81	317.42	318.87	319.87	319.43	318.01	315.74	314.00	313.68	314.84	316.03
1961	316.73	317.54	318.38	319.31	320.42	319.61	318.42	316.63	314.83	315.16	315.94	316.85
1962	317.78	318.40	319.53	320.42	320.85	320.45	319.45	317.25	316.11	315.27	316.53	317.53
1963	318.58	318.92	319.70	321.22	322.08	321.31	319.58	317.61	316.05	315.83	316.91	318.20
1964	319.41	320.07	320.74	321.40	322.06	321.73	320.27	318.54	316.54	316.71	317.53	318.55
1965	319.27	320.28	320.73	321.97	322.00	321.71	321.05	318.71	317.66	317.14	318.70	319.25
1966	320.46	321.43	322.23	323.54	323.91	323.59	322.24	320.20	318.48	317.94	319.63	320.87
1967	322.17	322.34	322.88	324.25	324.83	323.93	322.38	320.76	319.10	319.24	320.56	321.80
1968	322.40	322.99	323.73	324.86	325.40	325.20	323.98	321.95	320.18	320.09	321.16	322.74
1969	323.83	324.26	325.47	326.50	327.21	326.54	325.72	323.50	322.22	321.62	322.69	323.95
1970	324.89	325.82	326.77	327.97	327.91	327.50	326.18	324.53	322.93	322.90	323.85	324.96
1971	326.01	326.51	327.01	327.62	328.76	328.40	327.20	325.27	323.20	323.40	324.63	325.85
1972	326.60	327.47	327.58	329.56	329.90	328.92	327.88	326.16	324.68	325.04	326.34	327.39
1973	328.37	329.40	330.14	331.33	332.31	331.90	330.70	329.15	327.35	327.02	327.99	328.48
1974	329.18	330.55	331.32	332.48	332.92	332.08	331.01	329.23	327.27	327.21	328.29	329.41
1975	330.23	331.25	331.87	333.14	333.80	333.43	331.73	329.90	328.40	328.17	329.32	330.59
1976	331.58	332.39	333.33	334.41	334.71	334.17	332.89	330.77	329.14	328.78	330.14	331.52
1977	332.75	333.24	334.53	335.90	336.57	336.10	334.76	332.59	331.42	330.98	332.24	333.68
1978	334.80	335.22	336.47	337.59	337.84	337.72	336.37					
1979	336.05	336.59	337.79	338.71	339.30	339.12	337.56					
1980	337.84	338.19	339.91	340.60	341.29	341.00	339.39					
1981	339.06	340.30	341.21	342.33	342.74	342.08	340.32					
1982	340.57	341.44	342.53	343.39	343.96	343.18	341.88					
1983	341.20	342.35	342.93	344.77	345.58	345.14	343.81					
1984	343.52	344.33	345.11	346.88	347.25	346.62	345.22					
1985	344.79	345.82	347.25	348.17	348.74	348.07	346.38					
1986	346.11	346.78	347.68	349.37	350.03	349.37	347.76					

```
> library(dslabs)
> olive
```

	region	area	palmitic	palmitoleic	stearic	oleic	linoleic	linolenic	arachidic	eicosenoic
1	Southern Italy North-Apulia	10.75	0.75	2.26	78.23	6.72	0.36	0.60	0.29	
2	Southern Italy North-Apulia	10.88	0.73	2.24	77.09	7.81	0.31	0.61	0.29	
3	Southern Italy North-Apulia	9.11	0.54	2.46	81.13	5.49	0.31	0.63	0.29	
4	Southern Italy North-Apulia	9.66	0.57	2.40	79.52	6.19	0.50	0.78	0.35	
5	Southern Italy North-Apulia	10.51	0.67	2.59	77.71	6.72	0.50	0.80	0.46	
6	Southern Italy North-Apulia	9.11	0.49	2.68	79.24	6.78	0.51	0.70	0.44	
7	Southern Italy North-Apulia	9.22	0.66	2.64	79.90	6.18	0.49	0.56	0.29	
8	Southern Italy North-Apulia	11.00	0.61	2.35	77.28	7.34	0.39	0.64	0.35	
9	Southern Italy North-Apulia	10.82	0.60	2.39	77.45	7.09	0.46	0.83	0.33	
10	Southern Italy North-Apulia	10.37	0.55	2.13	79.44	6.33	0.26	0.52	0.30	
11	Southern Italy North-Apulia	10.51	0.35	2.19	79.78	6.05	0.21	0.65	0.24	
12	Southern Italy North-Apulia	10.36	0.59	2.35	78.68	6.61	0.30	0.62	0.44	
13	Southern Italy North-Apulia	10.74	0.70	2.14	77.28	7.47	0.50	0.79	0.33	
14	Southern Italy North-Apulia	8.75	0.52	2.43	80.18	6.55	0.41	0.79	0.32	
15	Southern Italy North-Apulia	9.52	0.49	2.54	77.95	7.80	0.50	0.75	0.41	
16	Southern Italy North-Apulia	11.55	0.98	2.01	76.06	8.16	0.32	0.60	0.29	
17	Southern Italy North-Apulia	9.43	0.94	1.83	78.40	7.88	0.42	0.75	0.31	
18	Southern Italy North-Apulia	12.78	0.69	2.05	73.44	9.57	0.45	0.70	0.28	
19	Southern Italy North-Apulia	9.61	0.70	1.95	79.58	7.42	0.46	0.75	0.30	
20	Southern Italy North-Apulia	9.52	0.77	2.58	78.20	7.36	0.43	0.78	0.33	

```
> library(tidyverse)
> fruit
```

[1] "apple"	"apricot"	"avocado"	"banana"	"bell pepper"
[6] "bilberry"	"blackberry"	"blackcurrant"	"blood orange"	"blueberry"
"boysenberry"	"breadfruit"	"canary melon"	"cantaloupe"	"cherimoya"
"cherry"	"chili pepper"	"clementine"	"cloudberry"	"coconut"
"cranberry"	"cucumber"	"currant"	"damson"	"date"
"dragonfruit"	"durian"	"eggplant"	"elderberry"	"feijoa"
"fig"	"goji berry"	"gooseberry"	"grape"	"grapefruit"
"guava"	"honeydew"	"huckleberry"	"jackfruit"	"jambul"
"jujube"	"kiwi fruit"	"kumquat"	"lemon"	"lime"
"loquat"	"lychee"	"mandarine"	"mango"	"mulberry"
"nectarine"	"nut"	"olive"	"orange"	"pamelo"
"papaya"	"passionfruit"	"peach"	"pear"	"persimmon"
"physalis"	"pineapple"	"plum"	"pomegranate"	"pomelo"
"purple mangosteen"	"quince"	"raisin"	"rambutan"	"raspberry"
"redcurrant"	"rock melon"	"salal berry"	"satsuma"	"star fruit"
"strawberry"	"tamarillo"	"tangerine"	"ugli fruit"	"watermelon"





In this lecture

- Introduction to R and RStudio
- Basics of R programming
- **Data types & data structures**
- Basic operations



Atomic data types

```
> a <- 3
> class(a)
[1] "numeric"
> b <- 3:3
> class(b)
[1] "integer"
> c <- "3"
> class(c)
[1] "character"
```

- Variables in R can be of different types. Use the function `class` for help.
- (Major) atomic data types in R:
 - `numeric` (float/double)
 - `integer` (byte, short, int, long, etc.)
 - `logical` (bool)
 - `character` (string)
 - `factor`, will be covered later



Atomic data types

What's the difference between a numeric and an integer in R?

```
> class(1)
[1] "numeric"
> class(1L)
[1] "integer"
> class(as.integer(1))
[1] "integer"
```

```
> is.integer(1)
[1] FALSE
> is.integer(1L)
[1] TRUE
> is.numeric(1)
[1] TRUE
> is.numeric(1L)
[1] TRUE
> is.double(1)
[1] TRUE
> is.double(1L)
[1] FALSE
```

- In most cases they are interchangeable, automatically coerced
- `integer` is mostly used for indexing

```
> c(1, 2, 3, 4)[1.99] == c(1, 2, 3, 4)[1]
[1] TRUE
```

<https://stackoverflow.com/questions/23660094/whats-the-difference-between-integer-class-and-numeric-class-in-r>

data.frame

```
> library(dslabs)
> data(murders)
> class(murders)
[1] "data.frame"
> head(murders, n=3)
  state abb region population total
1 Alabama AL  South   4779736   135
2  Alaska AK   West    710231    19
3 Arizona AZ   West   6392017   232
```

The most common data structure in R

A 2D table with

- Rows: observations
- Columns: different variables reported for each observation, can be of different data types

Environment	History	Connections	Tutorial
  Import Dataset ▾  159 MiB ▾ 			
R ▾	 Global Environment ▾		
Data			
▼ murders		51 obs. of 5 variables	
\$ state		: chr "Alabama" "Alaska" "Arizona" "Arkansas" ...	
\$ abb		: chr "AL" "AK" "AZ" "AR" ...	
\$ region		: Factor w/ 4 levels "Northeast","South",...: 2 4 4 2 4 4 1 2 2 2 ...	
\$ population		: num 4779736 710231 6392017 2915918 37253956 ...	
\$ total		: num 135 19 232 93 1257 ...	

The "murders" dataset

```
> ?murders
```

US gun murders by state for 2010

Description

Gun murder data from FBI reports. Also contains the population of each state.

Usage

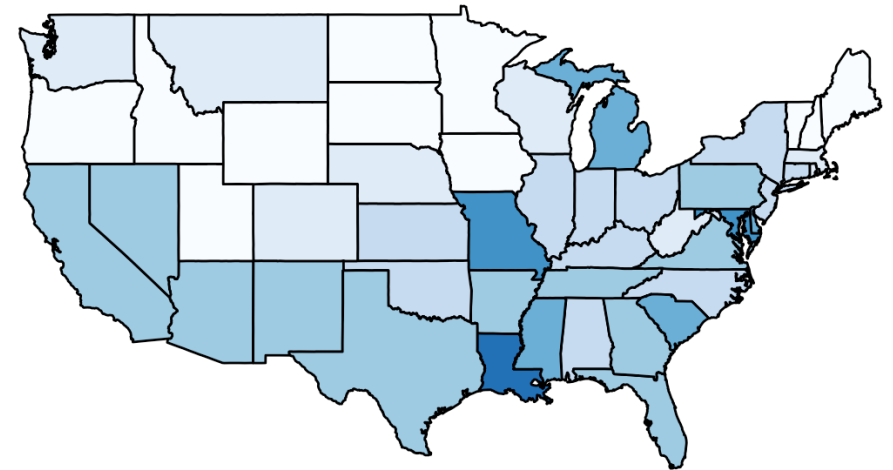
```
murders
```

Format

An object of class "data.frame".

Details

- state. US state
- abb. Abbreviation of US state
- region. Geographical US region
- population. State population (2010)
- total. Number of gun murders in state (2010)





Examine a `data.frame`

Use: `?`, `str`, `head`, environment panel of RStudio

```
> str(murders)
```

```
'data.frame':  51 obs. of  5 variables:
 $ state      : chr  "Alabama" "Alaska" "Arizona" "Arkansas" ...
 $ abb        : chr  "AL" "AK" "AZ" "AR" ...
 $ region     : Factor w/ 4 levels "Northeast","South",...: 2 4 4 2 4 4 1 2 2 2 ...
 $ population: num  4779736 710231 6392017 2915918 37253956 ...
 $ total      : num  135 19 232 93 1257 ...
```

```
> head(murders)
```

	state	abb	region	population	total
1	Alabama	AL	South	4779736	135
2	Alaska	AK	West	710231	19
3	Arizona	AZ	West	6392017	232
4	Arkansas	AR	South	2915918	93
5	California	CA	West	37253956	1257
6	Colorado	CO	West	5029196	65

Print the first six lines instead of print all lines.

What if I want to print 10 lines?
`?head`



The accessor: \$

To access the different variables/columns: use the accessor operator \$

```
> murders$population
[1] 4779736 710231 6392017 2915918 37253956 5029196 3574097 897934 601723
[10] 19687653 9920000 1360301 1567582 12830632 6483802 3046355 2853118 4339367
[19] 4533372 1328361 5773552 6547629 9883640 5303925 2967297 5988927 989415
[28] 1826341 2700551 1316470 8791894 2059179 19378102 9535483 672591 11536504
[37] 3751351 3831074 12702379 1052567 4625364 814180 6346105 25145561 2763885
[46] 625741 8001024 6724540 1852994 5686986 563626
>
```

But how did we know to use population?

Use `names` to reveal the available variable names

```
> names(murders)
[1] "state"      "abb"        "region"     "population" "total"
```

The order of the entries in `murders$population` **preserves the order of the rows** in the original data frame. This will later permit us to manipulate one variable based on the results of another

Data structure: vector

```
> pop <- murders$population  
> length(pop)  
[1] 51
```

```
> length(0)  
[1] 1
```

```
> class(pop)  
[1] "numeric"
```

```
> class(murders$state)  
[1] "character"
```

`murders$population` contains multiple numbers; we call these types of objects vectors.

- Function `length`
- A single number is technically also a vector, of length 1

`murders$population` is a numeric vector: entries are numbers.

Vectors can also be of class character.

All entries in a vector need to be of the same atomic data type

Data structure: vector

```
> z <- 3 == 2
> z
[1] FALSE
> class(z)
[1] "logical"
```

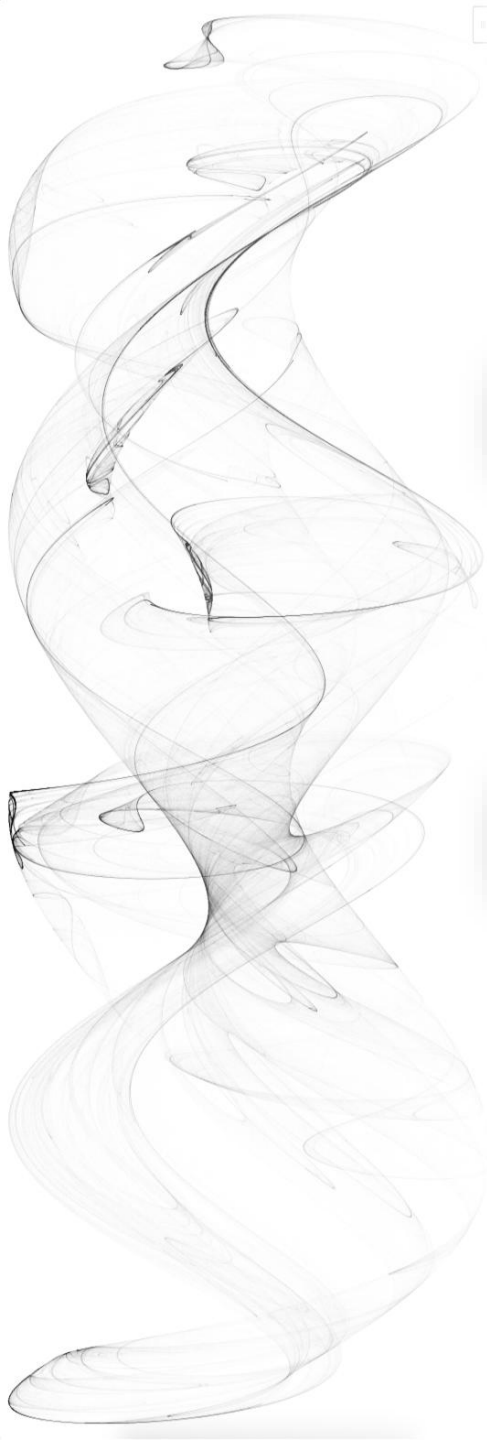
Logical vector contain entries that are either TRUE or FALSE (UPPER CASE).

== is a relational operator asking if 3 is equal to 2

= is for assigning/providing values to variables

```
> pop <- murders$population
> class(pop > 1000000)
[1] "logical"
```

? "==", ? ">", ? Comparison



Data structure: vector of factor

```
> class(murders$region)
[1] "factor"
```

`murders$region` is not a character vector, it is a factor vector

Factors are especially useful for storing categorical data. We can see that there are only 4 regions by using the `levels` function

```
> levels(murders$region)
[1] "Northeast"      "South"           "North Central"  "West"
```

In the background, R stores these factors of different levels as integers and keeps a map to keep track of the labels.

More memory efficient than storing characters



Data structure: vector of factor

Factor will have an intrinsic order of levels.

Specify an order through the `levels` argument when creating factor.

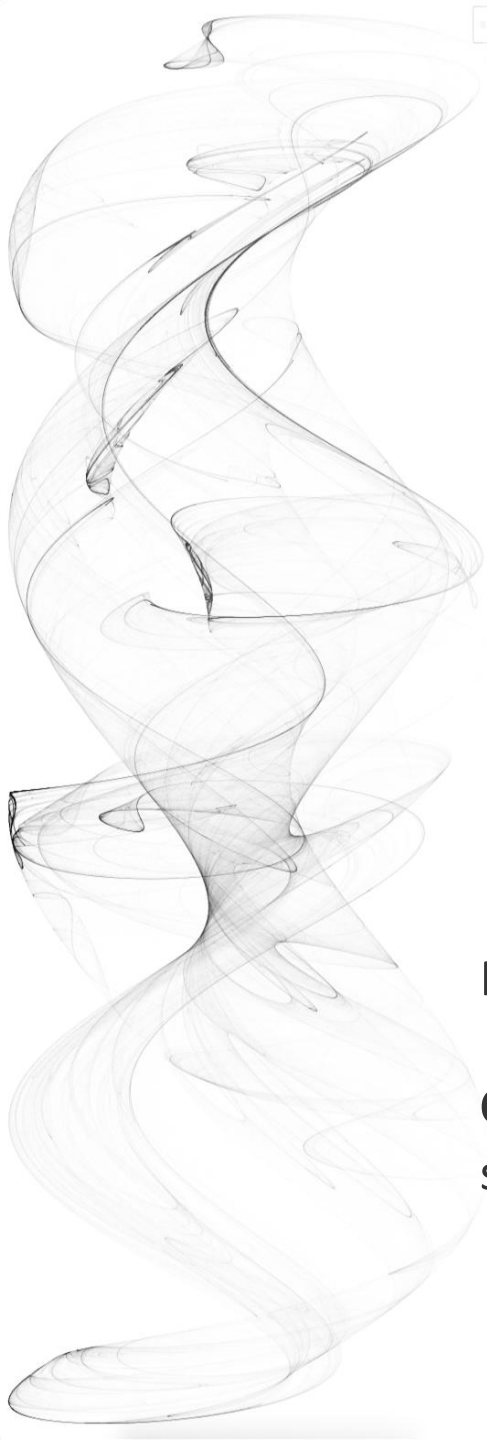
```
> factor(x="C", levels=c("C", "A", "B"))  
[1] C  
Levels: C A B
```

Function `reorder` lets us change the order of the levels of a factor variable.

This is very useful for organize data visualization

```
> region <- murders$region  
> value <- murders$total  
> region <- reorder(region, value, FUN = sum)  
> levels(region)  
[1] "Northeast"      "North Central"  "West"           "South"
```

Interpretation: states in the Northeast have the least murders and states in the South have the most



Data structure: define a vector with `c()`

```
> grades <- c(95, 82, 91, 97, 93)
> class(grades)
[1] "numeric"
> class(as.character(grades))
[1] "character"
> grades <- c(95, 82, 91, 97, "93")
> class(grades)
[1] "character"
> grades[[1]]
[1] "95"
```

Function `c` creates a vector (concatenate), which can only contain one type of data.

Coercion: when given different types to `c`, elements will be forced to have the same atomic data type

logical → integer → numeric → character

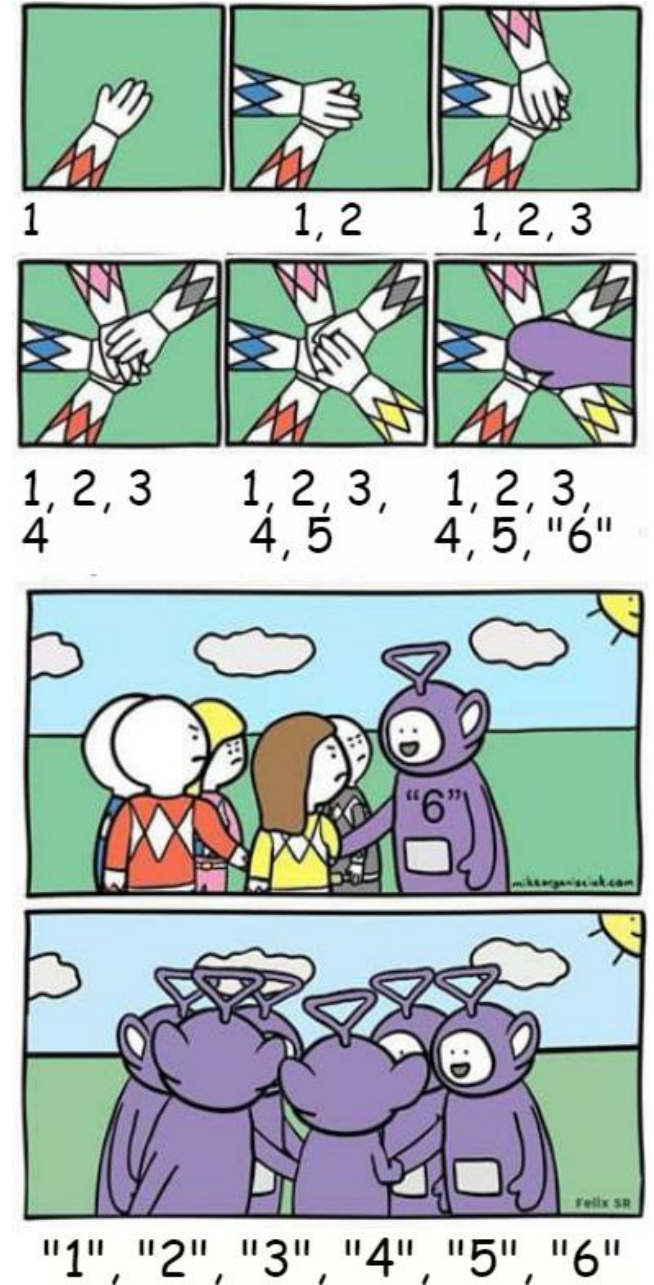
Coercion

```
> data(murders)
> head(murders, n=2)
  state abb region population total
1 Alabama AL  South    4779736   135
2 Alaska  AK   West     710231    19
> class(murders$population)
[1] "numeric"
> murders$population[2] <- "A lot"
> head(murders, n=2)
  state abb region population total
1 Alabama AL  South    4779736   135
2 Alaska  AK   West      A lot    19
> class(murders$population)
[1] "character"
```

```
> x <- c(1, "canada", 3)
> x
[1] "1"      "canada" "3"
> class(x)
[1] "character"
```

BE CAREFUL!

No warning is issued
in these examples,
could be sources of
errors/bugs.





Coercion

```
> x <- c("1", "b", "3")
> x <- as.numeric(x)
Warning message:
NAs introduced by coercion
> x[[2]]
[1] NA
> mean(x)
[1] NA
> mean(x, na.rm = TRUE)
[1] 2
```

When forcing character $\rightarrow$ numeric, NA is introduced

NAs (a logical value: Not Available) are common problem in real-world datasets: missing values, typos, etc.

Coercion

C/C++ and other
type-safe languages



mcc

@mcclure111

Follow

In C++ we don't say "Missing asterisk" we say "error C2664: 'void std::vector<block,std::allocator<_Ty>>::push_back(const block &)': cannot convert argument 1 from 'std::_Vector_iterator<std::_Vector_val<std::_Simple_types<block>>>' to 'block &&'" and i think that's beautiful

4:30 PM - 1 Jun 2018

292 Retweets 926 Likes



20

292

926



- Coercion is an attempt to be flexible with data types
- When an entry does not match the expected, some of the prebuilt R functions try to guess what was meant before throwing an error.
- This often leads to confusion. Failing to understand coercion can drive programmers crazy when attempting to code in R since it behaves quite differently from most other languages in this regard

Data structure: define a vector with `seq()`

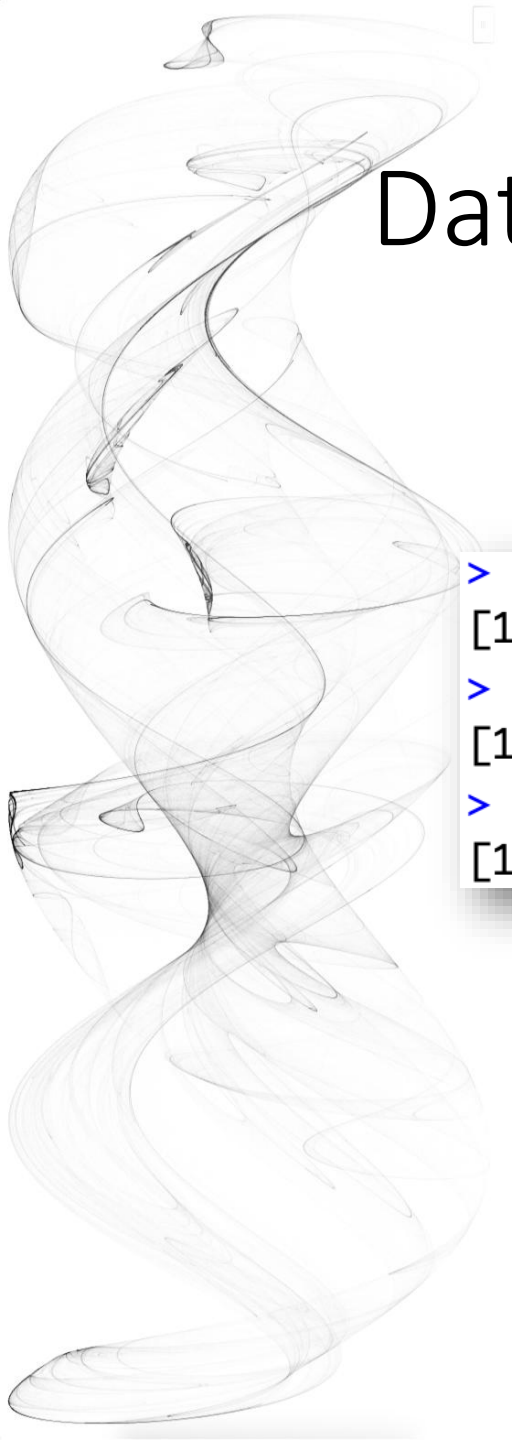
```
> seq(1,10)
[1] 1 2 3 4 5 6 7 8 9 10
> 1:10
[1] 1 2 3 4 5 6 7 8 9 10
```

Creating a vector with an integer/numeric sequence

- `seq` generates integer vector by default

Use argument `by` OR `length.out` to specify interval/length of the desired vector.

```
> seq(1, 10, by=2)
[1] 1 3 5 7 9
> seq(1, 10, length.out=6)
[1] 1.0 2.8 4.6 6.4 8.2 10.0
> seq(1, 10, by=2, length.out=6)
Error in seq.default(1, 10, by = 2, length.out = 6) : too many arguments
```

Data structure: define a vector with `rep()`

```
> rep(1, times=5)
[1] 1 1 1 1 1
> rep(seq(1:3), times=3)
[1] 1 2 3 1 2 3 1 2 3
> rep(seq(1:3), each=3)
[1] 1 1 1 2 2 2 3 3 3
```

Extend a vector for `n` times

Naming entries in a vector

```
> codes <- c(italy = 380, canada = 124, egypt = 818)
> codes
italy canada egypt
  380    124    818
```

```
> codes <- c("italy" = 380, "canada" = 124, "egypt" = 818)
> codes
italy canada egypt
  380    124    818
```

```
> class(codes)
[1] "numeric"
> names(codes)
[1] "italy" "canada" "egypt"
```

Sometimes it is useful to name the entries of a vector. For example, when defining a vector of country codes, we can use the names to connect the two

Quotes are optional

The object codes continues to be a numeric vector, but with names



Naming entries in a vector

```
> codes <- c(380, 124, 818)
> codes
[1] 380 124 818
> country <- c("italy", "canada", "egypt")
> names(codes) <- country
> codes
italy canada  egypt
  380    124    818
```

Names can be assigned later



Data structure: list

```
> record <- list(name = "Tai Man Chan",  
+               student_id = 1234,  
+               grades = c(95, 82, 91, 97, 93),  
+               final_grade = "A")
```

```
> record  
$name  
[1] "Tai Man Chan"  
  
$student_id  
[1] 1234  
  
$grades  
[1] 95 82 91 97 93  
  
$final_grade  
[1] "A"
```

List is DIFFERENT from a named vector;

Lists can contain arbitrary combinations of different data types.

- Data frames are a special case of lists.



Data structure: list

```
> record$student_id  
[1] 1234
```

Access component with the accessor \$

```
> record[["student_id"]]  
[1] 1234
```

Access with double square brackets [[]]

```
> record[["student_id"]]  
[1] 1234  
> record["student_id"]  
$student_id  
[1] 1234  
  
> class(record[["student_id"]])  
[1] "numeric"  
> class(record["student_id"])  
[1] "list"
```

Double brackets ([[]]) return the value.
Single bracket ([]) returns a list with the key and the value



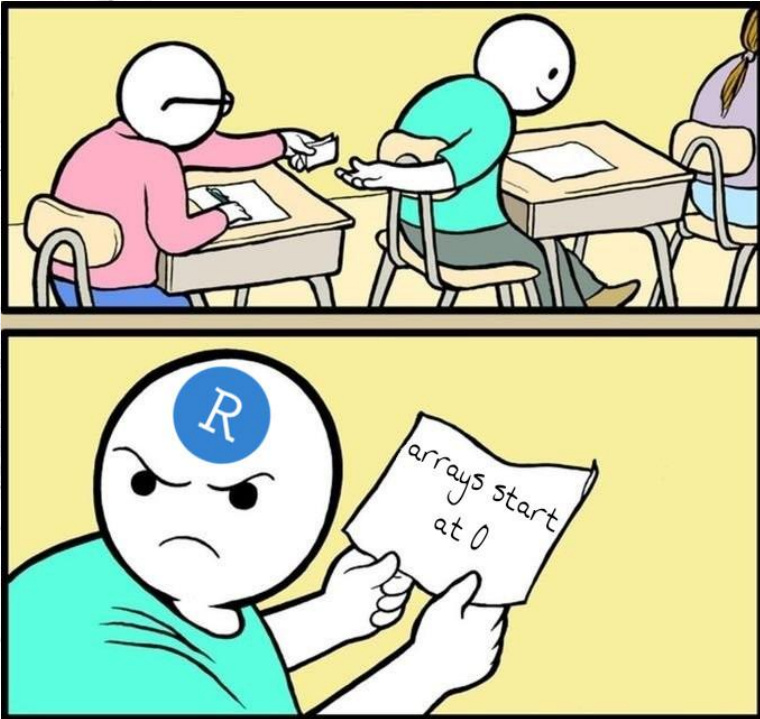
Data structure: indexing

Feature	[]	[[]]
Output Type	Same as input (e.g., list, vector)	Raw content (e.g., number, character)
Number of Elements	Can return multiple elements	Returns only one element
Hierarchy	Preserves structure	Drops one level (e.g., list → element)
Name-Based Subsetting	Works (<code>list["a"]</code>)	Works (<code>list[["a"]]</code> or <code>list\$a</code>)

Also see: `?["` `?Extract`

<https://stackoverflow.com/questions/1169456/the-difference-between-bracket-and-double-bracket-for-accessing-the-el>

Data structure: indexing



The tricky bit: indexing in R starts with 1

- Influence of statistical practice
- Inherited from FORTRAN

```
> record2[[1]]  
[1] "Tai Man Chan"  
> record2[[0]]  
Error in record2[[0]] :  
  attempt to select less than one element in get1index <real>
```

Data structure: indexing

```
> codes <- c(italy = 380, canada = 124, egypt = 818)
> codes[2]
canada
124
```

Use integer as index

```
> codes[c(1,3)]
italy egypt
380 818
```

Use a vector (of integers) as index

```
> codes[seq(1,2)]
italy canada
380 124
```

Use `seq` to define integer sequence for indexing

```
> codes["canada"]
canada
124
> codes[c("egypt","italy")]
egypt italy
818 380
```

If the elements have names, we can also use names as index



Data structure: index with 0

```
> v<-c(1,2,3)
> v[1]
[1] 1
> v[0]
numeric(0)
> class(v[1])
[1] "numeric"
> class(v[0])
[1] "numeric"
```

```
> l<-list(1,2,3)
> l[[1]]
[1] 1
> l[1]
[[1]]
[1] 1

> l[[0]]
Error in l[[0]] :
  attempt to select less than one element in get1index <real>
> l[0]
list()
```

Indexing with zero (`[0]`, not `[[0]]`) gives you the data type of the object

Very rarely used



Data structure: index with -1

```
> x <- c(1, 2, 3, 4, 5)
> x[-1]
[1] 2 3 4 5
> x[-2]
[1] 1 3 4 5
```

Indexing with negative number
(`[-1]`, not `[[-1]]`) gives the
same data structure, but with
the specified item excluded

```
> x <- list(item1 = 1, item2 = 2, item3 = 3)
> x[-1]
$item2
[1] 2

$item3
[1] 3

> x[-2]
$item1
[1] 1

$item3
[1] 3
```

See also: <https://stackoverflow.com/questions/7336679/in-r-what-does-a-negative-index-do>

Data structure: index with -1

Language	Index Base	-1 Meaning	Example	Notes
R	1-based	Exclude element at position 1 (drop by position)	<code>x <- 1:5; x[-1] # 2 3 4 5</code>	Cannot mix positive and negative indices. To get last element, use <code>x[length(x)]</code> or <code>tail(x, 1)</code> .
Python (lists, NumPy, pandas <code>.iloc</code>)	0-based	Last element (from end)	<code>a = [10, 20, 30]; a[-1] # 30</code>	Negative indices count from end: <code>-1</code> last, <code>-2</code> second last.
MATLAB	1-based	Invalid	<code>A(end)</code> for last	Negative indices not allowed. Use <code>end</code> , <code>end-1</code> for second last.
C++ (<code>std::vector</code> , arrays)	0-based	Invalid / UB	<code>v[-1]</code> → undefined behavior	Must use <code>v.back()</code> or <code>v[v.size()-1]</code> . No built-in negative indexing.
JavaScript	0-based	Not last by default; reads property <code>"-1"</code>	<code>arr.at(-1)</code> → last (ES2022+)	<code>arr[-1]</code> usually undefined. Use <code>.at(-1)</code> or <code>arr[arr.length - 1]</code> .



Data structure: matrix

- Matrix: two-dimensional data structure with rows and columns.
- All entries have to be of the same atomic data type
- Similar indexing method as data frame

matrix {base}

R Documentation

Matrices

Description

`matrix` creates a matrix from the given set of values.

`as.matrix` attempts to turn its argument into a matrix.

`is.matrix` tests if its argument is a (strict) matrix.

Usage

```
matrix(data = NA, nrow = 1, ncol = 1, byrow = FALSE,  
       dimnames = NULL)
```

```
as.matrix(x, ...)  
## S3 method for class 'data.frame'  
as.matrix(x, rownames.force = NA, ...)
```

```
is.matrix(x)
```

Arguments

<code>data</code>	an optional data vector (including a list or expression vector). Non-atomic classed R objects are coerced by as.vector and all attributes discarded.
<code>nrow</code>	the desired number of rows.
<code>ncol</code>	the desired number of columns.
<code>byrow</code>	logical. If <code>FALSE</code> (the default) the matrix is filled by columns, otherwise the matrix is filled by rows.
<code>dimnames</code>	A dimnames attribute for the matrix: <code>NULL</code> or a list of length 2 giving the row and column names respectively. An empty list is treated as <code>NULL</code> , and a list of length one as row names. The list can be named, and the list names will be used as names for the dimensions.
<code>x</code>	an R object.
<code>...</code>	additional arguments to be passed to or from methods.
<code>rownames.force</code>	logical indicating if the resulting matrix should have character (rather than <code>NULL</code>) rownames . The default, <code>NA</code> , uses <code>NULL</code> rownames if the data frame has 'automatic' row.names or for a zero-row data frame.

Data structure: matrix

```
> 1:12
[1] 1 2 3 4 5 6 7 8 9 10 11 12
> mat <- matrix(1:12, 4, 3)
> mat
      [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
[3,]    3    7   11
[4,]    4    8   12
```

```
> mat[2, 3]
[1] 10
```

```
> mat[2, ]
[1] 2 6 10
```

```
> mat[, 3]
[1] 9 10 11 12
```

```
> mat[2]
[1] 2
```

```
> mat[1:2,]
      [,1] [,2] [,3]
[1,]    1    5    9
[2,]    2    6   10
```

Define a matrix using the function `matrix`.

- numbers of rows and columns need to be specified

Access an entry by specifying row and column

Access the entire row/column, returns a vector

`mat[2]` gives you a single entry, not the second row/column

Access multiple rows/columns, returns a matrix

Data structure: matrix

```
> as.data.frame(mat)
```

	V1	V2	V3
1	1	5	9
2	2	6	10
3	3	7	11
4	4	8	12

Convert matrices into data frames using the function `as.data.frame`

```
> data("murders")
```

```
> class(murders)
```

```
[1] "data.frame"
```

```
> murders[25, 1]
```

```
[1] "Mississippi"
```

```
> murders[2:3, ]
```

	state	abb	region	population	total
2	Alaska	AK	West	710231	19
3	Arizona	AZ	West	6392017	232

Similar indexing method works for data frames

Data structure summary

Feature	Vector	List	Data Frame	Matrix
Homogeneous?	Yes (same type)	No (mixed types)	No (cols can vary)	Yes (same type)
Dimensions	1D	1D	2D (rows × cols)	2D (rows × cols)
Element Type	Atomic (e.g., numeric, character)	Any R object	Columns are vectors	Atomic (e.g., numeric, logical)
Creation	<code>c()</code> , <code>seq()</code> , etc.	<code>list()</code>	<code>data.frame()</code>	<code>matrix()</code>
Use Case	Simple sequences	Mixed/hierarchical data	Tabular data	Numeric computations



In this lecture

- Introduction to R and RStudio
- Basics of R programming
- Data types & data structures
- **Basic operations**

Sorting

```
> library(dslabs)
> data(murders)
> head(murders, n = 3)
  state abb region population total
1 Alabama AL  South   4779736   135
2  Alaska AK   West    710231    19
3 Arizona AZ   West   6392017   232
> sort(murders$total)
 [1]  2  4  5  5  7  8 11 12 12 16 19 21 22 27 32 36
[17] 38 53 63 65 67 84 93 93 97 97 99 111 116 118 120 135
[33] 142 207 219 232 246 250 286 293 310 321 351 364 376 413 457 517
[49] 669 805 1257
```

Let's try to gain some insights into the safety of different states in the context of gun murders by analyzing the `murders` dataset. Say we want to find the safest and the most dangerous state.

The function `sort` sorts a vector in ascending order.

The largest number of gun murders in a year in a single state is 1257.



Sorting

```
> index <- order(murders$total)
> index[1]
[1] 46
> murders$state[index[1:5]]
[1] "Vermont"      "North Dakota" "New Hampshire" "Wyoming"      "Hawaii"
> murders$state[index[(length(index) - 4):length(index)]]
[1] "Pennsylvania" "New York"     "Florida"      "Texas"        "California"
```

The function `order` takes a vector as input and returns the vector of indexes that sorts the input vector: the 46th entry has the least amount of gun murders

Vermont has the least amount of gun murders;
California has the most amount of gun murders;



Sorting

```
> x <- c(31, 4, 15, 92, 65)
> rank(x)
[1] 3 1 2 5 4
```

original	sort	order	rank
31	4	2	3
4	15	3	1
15	31	1	2
92	65	5	5
65	92	4	4

Although not as frequently used, the function `rank` is also related to order and can be useful.

For any given vector it returns a vector with the rank of the first entry, second entry, etc., of the input vector

- May serve as a data normalization technique

To summarize, let's look at the results of the three functions we have introduced



Sorting

```
> max(murders$total)
[1] 1257
> murders$state[which.max(murders$total)]
[1] "California"
> min(murders$total)
[1] 2
> murders$state[which.min(murders$total)]
[1] "Vermont"
```

Use `max` and `which.max` if we are only interested in the entry with the largest value.

Likewise, `min` and `which.min`

Does this mean California is the most dangerous state?

Likely no, why?



California had the most murders, but does this mean it is the most dangerous state? What if it just has many more people than any other state? We can quickly confirm that California indeed has the largest population:

```
> library(dslabs)
> data("murders")
> max(murders$population)
[1] 37253956
> which.max(murders$population)
[1] 5
> murders$state[which.max(murders$population)]
[1] "California"
```

How to account for population differences? Use murders per capita.

The reports we describe in the subsequent section used murders per 100,000 as the unit. To compute this quantity, we shall use vector arithmetic.



Vector arithmetic

In R, arithmetic operations on vectors occur element-wise. For a quick example, suppose we have height in inches, and we want to convert to centimeters. Notice what happens when we multiply inches by 2.54:

```
> inches <- c(69, 62, 66, 70, 70, 73, 67, 73, 67, 70)
> inches * 2.54
[1] 175.26 157.48 167.64 177.80 177.80 185.42 170.18 185.42 170.18 177.80
```

The average height of a Hong Kong male at 19-20 is 173.4cm, if for each entry we want to compute how many centimeters taller or shorter than the average we can subtract it from every entry like this:

```
> inches * 2.54 - 173.4
[1] 1.86 -15.92 -5.76 4.40 4.40 12.02 -3.22 12.02 -3.22 4.40
```

Vector arithmetic

If we have two vectors of the same length, arithmetic operators are calculated element-wise by default:

$$\begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix} + \begin{pmatrix} e \\ f \\ g \\ h \end{pmatrix} = \begin{pmatrix} a + e \\ b + f \\ c + g \\ d + h \end{pmatrix}$$

Let's compute the murder rates as:

```
> murder_rate <- murders$total / murders$population * 100000
> murder_rate
 [1]  2.8244238  2.6751860  3.6295273  3.1893901  3.3741383  1.2924531  2.7139722
 [8]  4.2319369 16.4527532  3.3980688  3.7903226  0.5145920  0.7655102  2.8369608
[15]  2.1900730  0.6893484  2.2081106  2.6732010  7.7425810  0.8280881  5.0748655
[22]  1.8021791  4.1786225  0.9992600  4.0440846  5.3598917  1.2128379  1.7521372
[29]  3.1104763  0.3798036  2.7980319  3.2537239  2.6679599  2.9993237  0.5947151
[36]  2.6871225  2.9589340  0.9396843  3.5977513  1.5200933  4.4753235  0.9825837
[43]  3.4509357  3.2013603  0.7959810  0.3196211  3.1246001  1.3829942  1.4571013
[50]  1.7056487  0.8871131
```

California is no longer near the top of the list:

```
> murders$abb[order(murder_rate)]
 [1] "VT" "NH" "HI" "ND" "IA" "ID" "UT" "ME" "WY" "OR" "SD" "MN" "MT" "CO" "WA" "WV"
[17] "RI" "WI" "NE" "MA" "IN" "KS" "NY" "KY" "AK" "OH" "CT" "NJ" "AL" "IL" "OK" "NC"
[33] "NV" "VA" "AR" "TX" "NM" "CA" "FL" "TN" "PA" "AZ" "GA" "MS" "MI" "DE" "SC" "MD"
[49] "MO" "LA" "DC"
```



Vector arithmetic - recycling

What if vectors of different lengths?

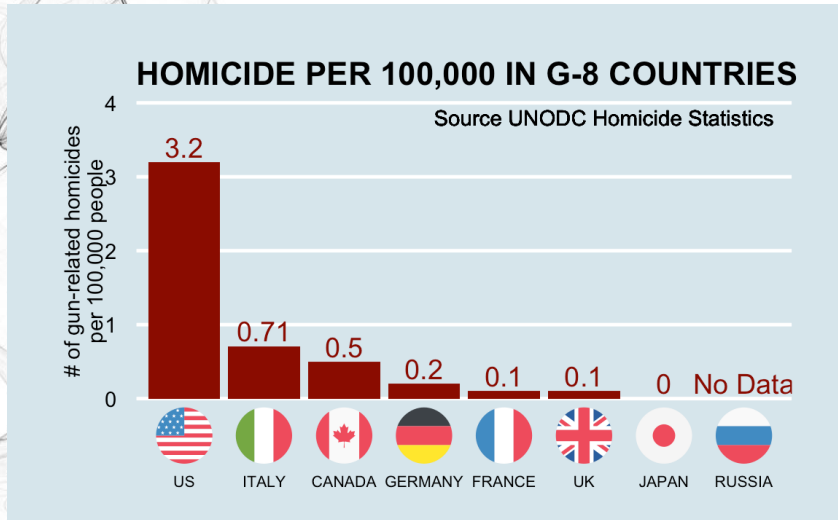
Recycling (another common source of errors/bugs): if the vectors don't match in length, the shorter one is automatically recycled (similar behavior as `rep`)

```
> x <- c(1, 2, 3)
> y <- c(10, 20, 30, 40, 50, 60, 70)
> x+y
[1] 11 22 33 41 52 63 71
Warning message:
In x + y : longer object length is not a multiple of shorter object length
```

We do get a warning, but no error

Indexing with a logical vector

Which states have less gun murders than Italy?



```
> murder_rate <- murders$total / murders$population * 100000
> ind <- murder_rate <= 0.71
> ind
[1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE
[14] FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[27] FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE
[40] FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE
> murders$state[ind]
[1] "Hawaii"      "Iowa"         "New Hampshire" "North Dakota" "Vermont"
> sum(ind)
[1] 5
```

Note that we get back a logical vector with TRUE for each entry smaller than or equal to 0.71.

To see which states these are, we can index the state vector with a logical vector.

When counting number of TRUEs, sum automatically coerced logical values to numeric: TRUE => 1, FALSE => 0

Logical operators

```
> True <- FALSE
> True & TRUE
[1] FALSE
```

<= Don't do this

```
> TRUE & TRUE
[1] TRUE
> c(TRUE, TRUE) & c(1, 0)
[1] TRUE FALSE
> TRUE && FALSE
[1] FALSE
> c(TRUE, TRUE) && c(1, 0)
Error in c(TRUE, TRUE) && c(1, 0) :
  'length = 2' in coercion to 'logical(1)'
```

TRUE is the logical value, not “true” or “True”.

& is “bitwise and”, can be applied to compare vectors
&& only looks at the first element

- and requires input of length 1, starting R 4.3.0

Always use & and |

Also see: ?"&" ?Logic

<https://stackoverflow.com/questions/6558921/boolean-operators-and>

which, match

```
> murders$state == "California"
[1] FALSE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[14] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[27] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[40] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
> which(murders$state == "California")
[1] 5
> murder_rate[which(murders$state == "California")]
[1] 3.374138
```

```
> murder_rate[which(murders$state == "California" | murders$state == "Florida")]
[1] 3.374138 3.398069
> murder_rate[which(murders$state == "Florida" | murders$state == "California")]
[1] 3.374138 3.398069
> murder_rate[match(c("California", "Florida"), murders$state)]
[1] 3.374138 3.398069
> murder_rate[match(c("Florida", "California"), murders$state)]
[1] 3.398069 3.374138
```

```
> murder_rate[which(murders$state == "Hong Kong" | murders$state == "California")]
[1] 3.374138
> murder_rate[match(c("Hong Kong", "California"), murders$state)]
[1]      NA 3.374138
```

Suppose we want to look up California's murder rate. For this type of operation, it is convenient to convert vectors of logical into indexes. The function `which` tells us which entries of a logical vector are `TRUE`

If instead of just one state we want to find out the murder rates for several states, say New York and Florida, we can use the function `match`. Notice that `match` output indices in the same order as the input vector

`match` always return a vector of same length, even with cases of mismatch



`%in%`

If rather than an index we want a logical vector that tells us whether or not each element of a first vector is in a second, we can use the function `%in%`:

```
> c("Boston", "Dakota", "Washington") %in% murders$state  
[1] FALSE FALSE  TRUE
```

The following operations are (almost) equivalent

```
> vec2 <- murders$state  
> vec1 <- c("Boston", "Dakota", "Washington")  
> match(vec1, vec2)  
[1] NA NA 48  
> which(vec2 %in% vec1)  
[1] 48
```

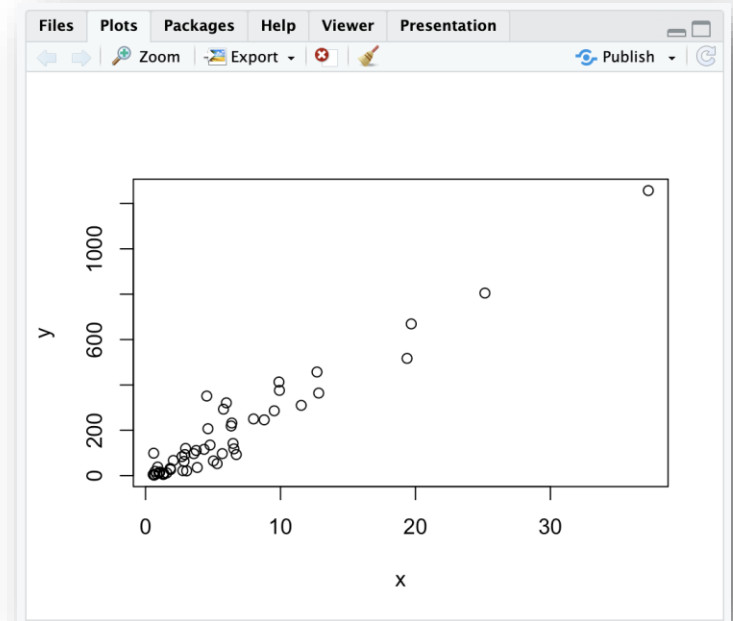
Basic plots – scatter plot

```
> x <- murders$population / 10^6  
> y <- murders$total  
> plot(x, y)
```



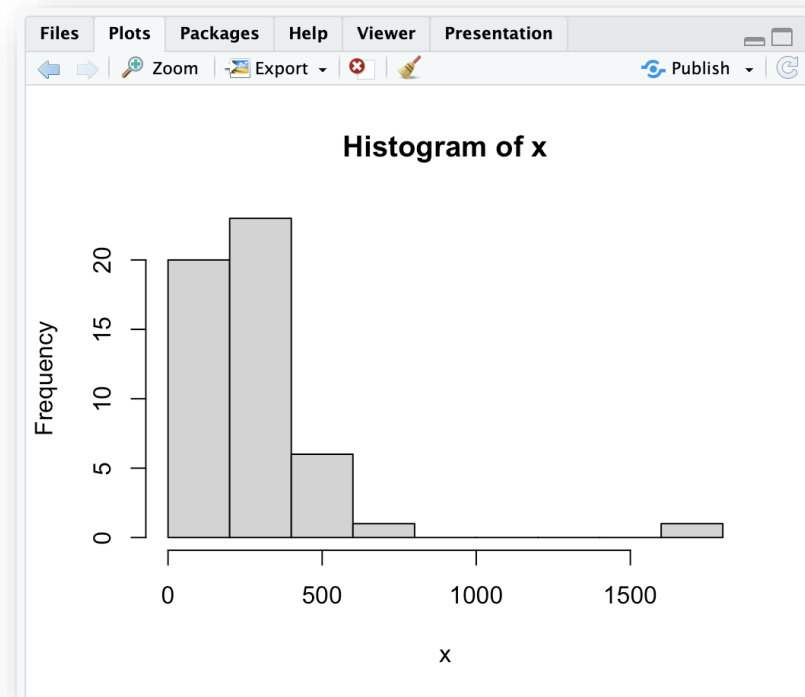
which is equivalent to this:

```
> with(murders, plot(population / 10^6, total))
```



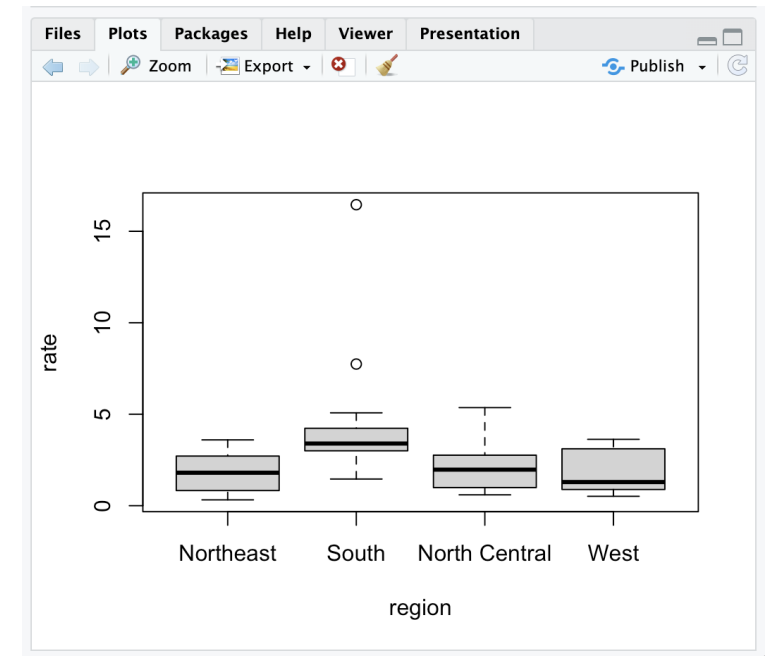
Basic plots - histogram

```
> with(murders, hist(total / population * 10^6))
```



Basic plots – box plot

```
> murders$rate <- with(murders, total / population * 100000)  
> boxplot(rate~region, data = murders)
```



Also see: `? "~"`

<https://stackoverflow.com/questions/14976331/use-of-tilde-in-r-programming-language>



In this lecture

- Introduction to R and RStudio
 - RStudio workspace
 - Environment setup, install and load packages: `tidyverse` `dslabs`
- Basics of R programming
 - Objects: `<-` (left assign), `=` (equal sign), environment panel
 - Built-in functions & operators: how to get documentation? Arguments?
 - Built-in datasets: `data()`
- Data types & data structures
 - numeric, integer, character, logical, factor (`levels` and `reorder`)
 - `data.frame`, vector, list, matrix
 - Indexing with integers, names, integer vectors: `$`, `[]`, `[][]`
- Basic operations
 - Sorting: `sort`, `order`, `rank`, `max`, `min`, `which.max`, `which.min`
 - Vector arithmetic
 - Indexing with logical vectors: `which`, `match`, `%in%`
 - Basic plots